



UNIVERSIDADE FEDERAL DO ABC – UFABC
CENTRO DE ENGENHARIA, MODELAGEM E CIÊNCIAS SOCIAIS APLICADAS
(CECS)
BACHARELADO EM ENGENHARIA BIOMÉDICA

Paulo César Menegon de Castro

**Desenvolvimento e Implementação de Sistema de Comunicação sem
Fio para Cadeira de Rodas Servo Assistida**

São Bernardo do Campo

2026

Paulo César Menegon de Castro

**Desenvolvimento e Implementação de Sistema de Comunicação sem Fio
para Cadeira de Rodas Servo Assistida**

Trabalho de Graduação para a conclusão do
curso de Bacharelado em Engenharia
Biomédica pela Universidade Federal do ABC,
desenvolvido no Centro de Engenharia,
Modelagem, Ciências Sociais Aplicadas
(CECS) como requisito parcial para a obtenção
do Título de Bacharel em Engenharia
Biomédica.

Orientador(a): Prof. Dr. Olavo Luppi Silva

São Bernardo do Campo

2026

1. INTRODUÇÃO

O Censo 2022 do IBGE (IBGE, 2022) revelou que 14,4 milhões de pessoas de 2 anos ou mais (7,3% da população) têm alguma deficiência no Brasil, com maior prevalência entre mulheres e idosos, sendo a deficiência visual a mais comum (7,9 milhões). O levantamento inédito também contabilizou 2,4 milhões de pessoas com transtorno do espectro autista.

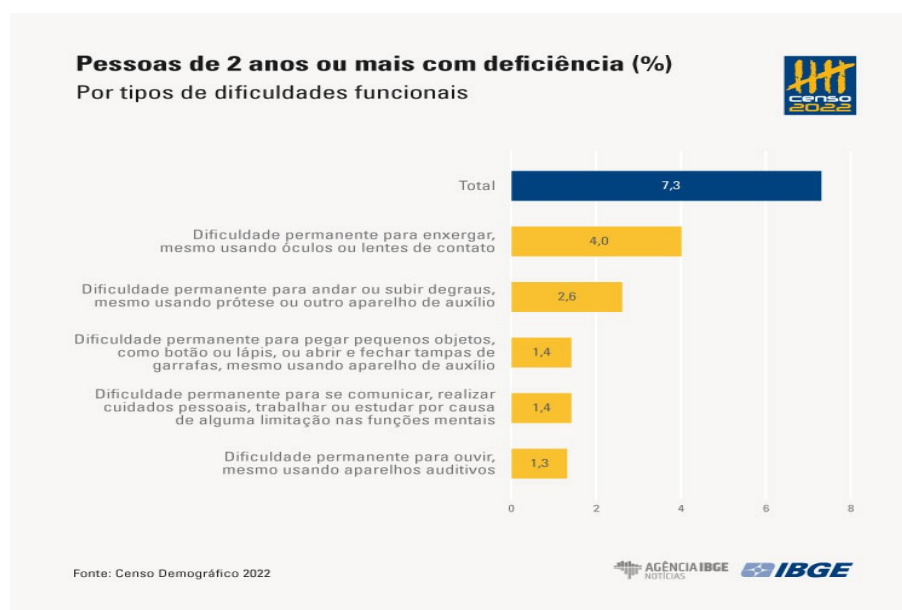
Segundo o Censo 2022:

- Distribuição das Dificuldades: Após a visual, as mais comuns são locomoção (5,2 milhões), motora fina (2,7 milhões) e auditiva (2,6 milhões).
- Domicílios: 16% dos domicílios no Brasil possuem pelo menos um morador com deficiência.
- Educação: A taxa de analfabetismo entre pessoas com deficiência é de 21,3%, quatro vezes maior que a das pessoas sem deficiência (5,2%).
- Trabalho: Dados da PNAD Contínua 2022 indicaram que apenas 29,2% das pessoas com deficiência participam da força de trabalho, contra 66,4% das pessoas sem deficiência, com alta taxa de informalidade (55%).
- Autismo: 1,2% da população (2,4 milhões) tem diagnóstico de TEA, com maior prevalência entre homens (1,5%).
- Regional: O Nordeste tem o maior percentual de domicílios com pessoas com deficiência (19,5%), seguido pelo Norte (17,8%).

Dentre os dados exibidos acima, nota-se que a deficiência de locomoção é a segunda deficiência mais frequente no país. A melhoria da locomoção destes indivíduos pode ser feita através do uso de alguma tecnologia assistiva. A tecnologia assistiva consiste em uma ampla gama de equipamentos, serviços, estratégias e práticas concebidas e aplicadas para minorar os problemas funcionais encontrados pelos indivíduos com deficiências (COOK, 2015). Entre esses recursos estão as cadeiras de rodas, que maximizam a funcionalidade por meio de estabilidade, alinhamento e conforto na postura sentada (SIMPSON, 2008), permitindo aos indivíduos completarem tarefas diárias com maior independência e acessar ambientes como a escola, o trabalho e a comunidade (COOPER, 2006).

As cadeiras de rodas podem ser manuais, quando a mobilidade depende das habilidades manuais ou de associação entre essas e auxílio de um pé para propulsão ou da assistência de terceiros para empurrar o recurso.

Figura 1 – Pessoas portadoras de deficiência no Brasil em 2022



Fonte: IBGE, 2022

Já as cadeiras de rodas motorizadas são aquelas que dependem de uma unidade de motor e do acesso aos controles, como os joysticks (COOK, 2015). Mundialmente, as cadeiras de rodas manuais são o tipo mais utilizado (OMS, 2008), o que também é reportado em estudos com sujeitos com lesões medulares (HETZ, 2008).

Para a OMS, por um lado, as cadeiras de rodas são um meio de as pessoas com deficiência exercerem seus direitos humanos e atingirem a inclusão e a igualdade de participação, contribuindo para a saúde, qualidade de vida e vida plena e ativa na comunidade (OMS, 2008). Por outro lado, as barreiras ambientais relacionadas à acessibilidade ainda são um grande desafio para os usuários deste recurso, já que dificultam a locomoção e interferem nas experiências sociais (MILLER, 2012). Dentre as barreiras mais comuns estão as rampas e degraus (33,3%), além de dificuldades na mobilidade em ambientes externos (13,3%) e o transporte da cadeira de rodas (13,3%) (MEDOLA, 2014). Também foram citadas dificuldades no uso da cadeira de rodas em ambientes internos, sendo a cozinha o lugar de maior dificuldade (72%), no uso de transporte público (59%) e particular (64%) (MEDOLA, 2014). Para a superação destas barreiras, as cadeiras de rodas motorizadas apresentam vantagens sobre as cadeiras manuais pois permitem maiores deslocamentos, com menor esforço por parte do usuário e sem depender de terceiros na maioria das situações do dia a dia.

O uso de motores elétricos em sistema de mobilidade é realidade estabelecida e são encontrados em aplicações tais como no transporte de carga, bicicletas, locomoção de pessoas com mobilidades reduzidas e muitas outras (MONTEIRO, 2016). O uso de um motor elétrico, adaptado ao sistema, pode substituir a força humana totalmente ou complementar essa força na forma de uma assistência. Essa assistência pode ser pré-programada ou mais elaborada no sentido de prover mais segurança e conforto ao operador, adaptando-se à sua solicitação (MONTEIRO, 2016). A assistência pré-programada pode ser vista no sentido de o motor sempre atuar da mesma forma, independentemente da atuação do operador, normalmente aplicando uma força (torque) constante assim que habilitada. A assistência que se adapta à condição operacional pode ter várias características como, por exemplo, ser função da declividade do movimento criado pelo operador (MONTEIRO, 2016).

A cadeira de rodas servo assistida, do inglês Pushrim-Activated Power Assisted Wheelchair (PAPAW) é uma tecnologia assistiva que desenvolve os princípios da cadeira de rodas e permite com que pessoas que possuem dificuldades em propulsioná-la manualmente, seja devido a alguma condição de saúde como obesidade, dores ao realizar o movimento de propulsão, baixas reservas cardiopulmonares, inabilidade de manter uma postura para a propulsão ou por não possuírem força suficiente no braço, possam se locomover sem a necessidade de soluções completamente elétricas (COOPER; COOPER; BONINGER, 2008).

A cadeira é construída através da modernização de cadeiras manuais, com o uso de hardware customizável e de rodas motorizadas, muitas vezes pensadas de modo que seja possível removê-las sem o uso de ferramentas (COOPER; COOPER; BONINGER, 2008). A PAPAW utiliza motores e uma bateria para aumentar a potência aplicada pelo usuário em algum dos aros da cadeira nos momentos de frenagem e de aceleração, e seu acionamento depende da entrada do usuário, que se configura como um torque realizado em algum dos aros da cadeira de rodas. Segundo o autor, o torque aplicado é amplificado pelos motores e por um sistema de engrenagens. Vale ressaltar que é utilizado um microcontrolador para cada uma das rodas traseiras do equipamento para controlar cada uma delas (COOPER et al., 2001; COOPER; COOPER; BONINGER, 2008).

Uma das alternativas para implementar o comportamento de servo-assistência na PAPAW é a do uso do controle por impedância. A técnica foi apresentada por (HOGAN, 1984) para lidar com tarefas de interação mecânica que incorporam processos de contato, nos quais a interação dinâmica não pode ser desprezada.

Exemplos dados pelo autor são situações nas quais são utilizadas ferramentas, como o uso de uma furadeira ou atividades realizadas por pessoas utilizando próteses. Em outras palavras, o controle por impedância é um sistema de controle no qual não se pode tratar o manipulador e o ambiente de forma isolada, pois o sistema de controle sofre ou exerce trabalho no ambiente. Assim, se procura controlar a relação entre duas grandezas em vez de levar em consideração uma única variável como ocorre em métodos convencionais de controle, o que faz com que a técnica de controle por impedância possua como principal mérito a capacidade de relacionar forma efetiva o controle das posições e força de contrato ao mesmo tempo (CUERVA, 2017; SONG; YU; ZHANG, 2017).

A construção de uma PAPA W é uma tarefa complexa, que requer conhecimento técnico em diversas áreas da engenharia e da saúde. O presente trabalho é uma continuação de um trabalho anterior (COQUEIRO, 2024) e tem o objetivo de construir um circuito de condicionamento do sinal de força tangencial exercido pelo usuário no aro de cadeira de rodas e transmitir os dados para um receptor localizado no módulo de controle. A necessidade de transmissão de dados sem o uso de cabos se faz necessária pelo fato de que os sensores estão localizados em uma parte móvel da cadeira de rodas e o módulo de controle ficará na parte estacionária da cadeira.

2. OBJETIVOS

O presente trabalho apresenta os seguintes objetivos:

1. Desenvolver um sistema de condicionamento de sinal capaz de realizar a transdução de sinal de força realizada no momento de propulsão de um aro de uma cadeira de rodas;
2. Desenvolver um sistema de comunicação sem fio bidirecional, associado a um microcontrolador a fim de realizar a conversão analógico digital do sinal condicionado e transmiti-lo para um receptor localizado no módulo de controle;
3. Construção de algoritmos para permitir que os dados sejam transmitidos e recebidos com baixa latência e sem perda significativa de pacotes.

3. COMUNICAÇÃO SEM FIO

As faixas de rádio ISM (Industrial, Scientific and Medical) são faixas de radiofrequência específicas designadas pela ITU Radiocommunication Sector e órgãos nacionais para equipamentos industriais, científicos e médicos, e não para comunicação. Essas faixas não licenciadas – geralmente 2,4 GHz, 915 MHz e 433 MHz – são amplamente utilizadas por Wi-Fi e Bluetooth, que precisam tolerar interferências de outros usuários.

Não há necessidade de pagamento de taxas para o uso dessas faixas de frequência, bastando respeitar um limite de potência de transmissão (geralmente inferior a 1 W) e não causar interferência em outras faixas de frequência. A regulamentação das bandas de frequência ISM varia entre os países. Por exemplo, nos Estados Unidos existem três faixas de frequência: 902–928 MHz, 2400–2484,5 MHz e 5725–5850 MHz, enquanto na Europa parte da faixa de 900 MHz é utilizada para comunicações GSM.

A faixa de 2.4 GHz é a faixa de frequência ISM comum a todos os países. Portanto, redes sem fio LAN (IEEE 802.11b/IEEE 802.11g), Bluetooth, ZigBee e outras podem operar na faixa de frequência de 2.4 GHz.

As bandas ISM comumente usadas são:

- 433 MHz (433.05 – 434.79 MHz): Usada para dispositivos de baixa potência, proporcionando maior alcance e penetração.
- 868 MHz (863 – 870 MHz): Usada na Europa como substituta da banda de 915 MHz.
- 915 MHz (902 – 928 MHz): Comumente usada na América do Norte para dispositivos celulares, RFID e IoT industrial.
- 2.4 GHz (2.400 – 2.500 GHz): Usada mundialmente para Bluetooth, Wi-Fi (IEEE 802.11b/g/n) e Zigbee.
- 5.8 GHz (5.725 – 5.875GHz): Usada para comunicação de alta velocidade e, frequentemente, para transmissão de vídeo.

Algumas tecnologias que utilizam faixas de rádio definidas pela ISM serão descritas a seguir.

3.1 Sigfox

A Sigfox é uma empresa francesa e uma rede de menor consumo e capacidade elevada direcionada para IoT, sendo uma das empresas pioneiras voltada para IoT.

Em linhas gerais, a tecnologia Sigfox disponibiliza uma alternativa de comunicação sem fio por meio de um software que diminui tanto o consumo de energia quanto o valor gasto dos aparelhos conectados. A banda de frequência em que opera é de 915 a 928 MHz e a largura de banda do canal de 100 MHz. Além disso, a Sigfox sustenta a banda ultra estreita e opera em espectro não licenciado, em que é gerada uma forma celular de comunicação de rede para indicar uma solução de menor rendimento em IoT (CHETTRI; BERA, 2020).

3.2 LoRa e LoRaWAN

A rede sem fio LoRaWAN altamente popular entre a LPWAN (Low Power Wide Area Network) (SIKIMÍĆ et al., 2020) ou protocolo LoRaWAN™ estabelece a arquitetura e a comunicação de um sistema e, para determinar esses parâmetros, o protocolo utiliza a tecnologia LoRa® (SILVA JUNIOR, 2016). A tecnologia LoRa é uma camada física responsável pela comunicação de longa distância. A rede de espectro LPWAN foi criada por “The Special Interest Group” (SIG) (Grupo de Interesse Especial). Trata-se de uma camada de enlace “Media Access Control” (MAC) (Controle de Acesso ao Meio) de um determinado modelo OSI, Open System Interconnection (Sistemas Abertos de Interconexão). Em linhas gerais, enquanto a rede LoRaWAN é um protocolo estrutural que permite a comunicação do sistema de rede, o LoRa serve para passagem de troca de dados de longo alcance (SIKIMÍĆ et al., 2020).

3.3 ZigBee

O ZigBee utiliza o LPWAN para promover sua tecnologia na área de IoT, destacando-se por possuir todas as camadas OSI e ser uma continuidade do IEEE.802.15.4 (CHETTRI; BERA, 2020). Assim, é comparada ao Bluetooth porque ambas são comunicações de curto alcance, já que o ZigBee transmite até 100m.

Destaque-se que o ZigBee utiliza microcontroladores de 8 bits (SIKIMÍĆ et al., 2020), e sua aplicabilidade ocorre na área de saúde, automação residencial e indústria relacionada a IoT (CHETTRI; BERA, 2020), e para tarefas em que não seja necessário transmitir dados em altas quantidades (SIKIMÍĆ et al., 2020)

3.4 Bluetooth

O Bluetooth é uma comunicação sem fio desenvolvida para promover a conexão entre aparelhos em curta distância. Essa tecnologia opera na banda de 2.4 GHz ISM (industrial, scientific and medical) e atualmente é o padrão aderido pela indústria de aparelhos móveis. Em adição, existe também o Bluetooth Low Energy, BLE que, além de consumir menos energia, tem capacidade de sustentar uma topologia estrelada ilimitada. Todavia, verifica-se que o BLE se aplica a dispositivos que utilizam fontes de energia autônomas e que possuem baixa largura de banda (SIKIMIĆ et al., 2020).

3.5 NB-IoT

O NB-IoT, Narrowband (A Internet de banda estreita das coisas), faz parte da LPWAN e surgiu através da tecnologia 3 GPP, 3rd Generation Partnership Project (Projeto de Parceria de 3.^a Geração). Opera em uma largura de banda de 180 kHz de uplink e downlink. Possui três modos de operação, sendo o primeiro “Operação Inband”, o segundo “Modo de Banda de Guarda” e o terceiro “Modo Autônomo”. Além disso, é uma tecnologia relacionada à aplicação para IoT, sendo utilizada em aplicativos de baixa potência e será de grande importância futuramente para as tecnologias que dependem da IoT (CHETTRI; BERA, 2020).

3.6. WLAN (Wi-Fi)

O Wi-Fi é uma tecnologia do padrão IEEE 802.11 e atua na faixa de 2,4 e 5GHz. Necessita de dois elementos para estabelecer uma rede central, sendo um ponto de acesso e um adaptador respectivamente (SIKIMIĆ et al., 2020). Ademais, a rede utilizada é a LAN (Local Area Network) e o Wi-Fi é recomendado para aplicação de curto alcance (CHETTRI; BERA, 2020). É importante saber que existe uma tecnologia mais eficiente para cada cenário em que a IoT necessite ser usada. Para determinar isso, existem dois conceitos conhecidos, como taxa de transmissão de dados (throughput) e alcance (distance).

A Tabela 1 resume algumas das características das redes sem fio apresentadas.

Tabela 1 – Comparação entre tecnologias de comunicação sem fio

Tecnologias	Alcance	Banda	Taxa de Transmissão	Consumo de Energia	Referências
LoRaWan	5km urbano e 20 km rural	915 e 928 MHz	50 kbps	Baixo	(QU et al., 2018) (LIYA e ARJUN, 2020)
SigFox	10 km urbano e 50 km rural	868 MHz e 915 e 928 MHz	100 kbps	Baixo	(QU et al., 2018) (LIYA e ARJUN, 2020)
NB-IoT	1km urbano e 10 km rural	700 e 800 e 900 MHz	20 kbps (FDMA - uplink), 200 kbps (OFDMA-downlink)	Baixo	(QU et al., 2018) (LIYA e ARJUN, 2020)
Wi-Fi	100m	2,4 GHz / 5 GHz / 6 GHz	9608 Mbps	Médio	(QU et al., 2018) (LIYA e ARJUN, 2020) (MIRAZ et al., 2015)
Bluetooth	10m	2402 MHz e 2480 MHz	2.2 Mbps	Baixo	(QU et al., 2018) (LIYA e ARJUN, 2020) (SANTOS et al., 2016)
Zigbee	De 10m a 100m	902 e 928 MHz, e 2,4 GHz	20, 40 kbps e 250 kbps	Baixo	(QU et al., 2018) (LIYA e ARJUN, 2020) (MIRAZ et al., 2015)

Fonte: Elaborado pelo autor

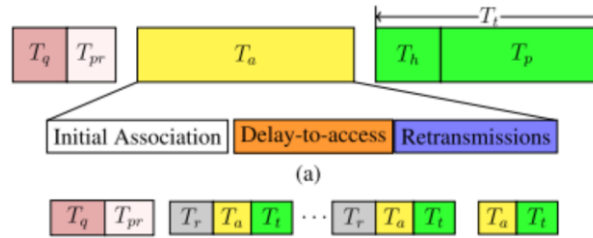
3.7. Latência

Latência pode ser definida como a duração de tempo entre a geração de um pacote e a sua correta recepção ao seu destinatário (JIANG et al., 2018). De acordo com os autores é difícil garantir uma latência total predefinida, dado que redes diferentes podem introduzir latências aleatórias os quais os valores exatos ou distribuições estatísticas não são tipicamente conhecidas inicialmente. No entanto, é determinado durante o artigo a formação de uma latência para um pacote que é enviado em um salto de rede.

- Os autores distinguem dois processos de latência em redes, sendo a latência se refere a um processo de codificação da mensagem denominada como latência de redes intranet, enquanto que quando a latência envolve outros computadores ou mais nós de rede é considerada como latência de internet. Para o caso do sensor utilizado, é de interesse apenas a latência de intranet, que será discutida nos parágrafos posteriores. Como é possível verificar da Figura X, os componentes mais importantes da latência são:
- O tempo de enfileiramento da informação, também conhecido como queueing latency (T_q);
- O tempo de processamento (T_{pr});
- O tempo levado por um pacote para acessar ao meio compartilhado, também chamado de medium access latency (T_a);

- O tempo de transmissão da informação (T_t), que é dado pela soma do tempo de envio do cabeçalho e metadados, conhecido por header, e da carga útil, também conhecida como payload (JIANG et al., 2018). Cada um dos tempos apresentados se refere a um processo.

Figura 2 – Componentes da latência

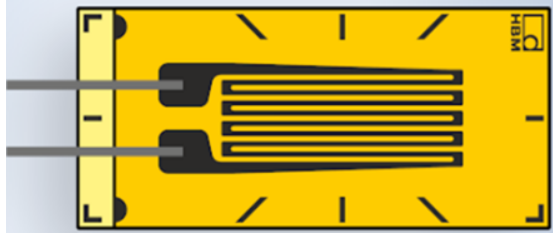


Fonte: JIANG et al., 2018

4. STRAIN GAUGE

O *strain gauge* (*strain gage*) ou extensômetro, é um sensor composto por um filme fino de metal (Figura 3) ou semiconductor que é colado na superfície do material que se deseja monitorar.

Figura 3 – *Strain gauge* LD 20 HBM



Fonte: HBM, 2026

Quando o material sofre alguma deformação, seja por alongamento ou compressão, o *strain gauge* também se deforma, o que altera a sua resistência elétrica. Essa mudança na resistência é proporcional à deformação sofrida, permitindo medir o quanto o material se expandiu ou contraiu.

A relação entre a deformação e a mudança de resistência é dada pelo *gauge factor*, e é descrita pela equação a seguir:

$$\frac{\Delta R}{R} = GF \cdot \frac{\Delta L}{L} \quad (1)$$

Onde: R é a resistência inicial do *strain gauge*,

ΔR é a variação de resistência do *strain gauge*,

L é o comprimento medido pelo *strain gauge*,

ΔL é a variação do comprimento medido pelo *strain gauge* e GF é o *gauge factor*.

Ao selecionar *strain gauges*, algumas das características a serem consideradas são:

- Resistência nominal e tolerância;
- Gauge factor e sua tolerância;

- *STC (self temperature compensation)*, onde o STC indica para qual tipo de material a variação de resistência possui compensação ao variar a temperatura da superfície na qual o *strain gauge* está instalado.

Para o modelo LD 20 da HBM (HBM, 2026) temos:

Figura 4 – Características técnicas do *strain gauge* LD 20 HBM

SPECIFICATIONS

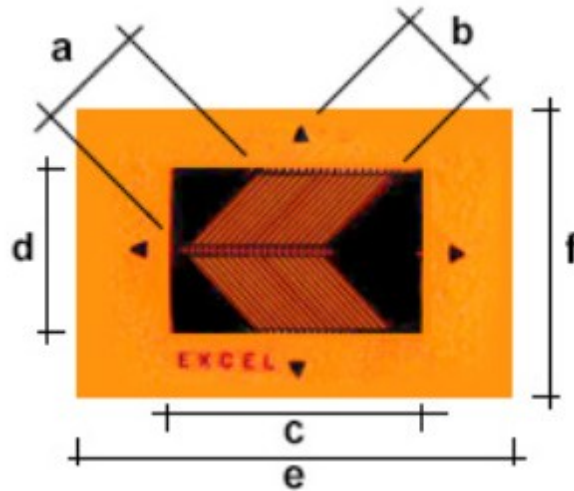
Strain gage construction		Foil strain gage with embedded measuring grid	
Measuring grid		Constantan	
Material		5	
Thickness	μm		
Carrier		Polyimide	
Material		45 ±15	
Thickness	μm		
Covering agent		Polyimide	
Material			
Connections		Solder tabs with strain relief made of copper-beryllium	
Leads		Nickel-plated copper leads, non-insulated, approx. 30 mm long	
Nominal (rated) resistance¹⁾	Ω	120 or 350	
Resistance tolerance¹⁾	%	±0.35	
Strain gage with leads			
Gage factor		approx. 2	
Nominal value of gage factor		Specified on each package	
Gage factor tolerance¹⁾	%	±1.0	
Temperature coefficient of gage factor¹⁾	1/K	approx. (115 ± 10) · 10 ⁻⁶	
Nominal value of gage factor temperature coefficient		Specified on each package	
Reference temperature	°C	23	
Operating temperature range		For static measurements (zero point related) For dynamic measurements (not zero point related)	
Strain gage with leads	°C	-70 ... +200 -200 ... +200	
Transverse sensitivity		Specified on each package	
at reference temperature when using CA80 adhesive	%	-0.1	
Temperature response		Specified on each package	
Temperature response as required, adapted to coefficients of thermal expansion			
α for ferritic steel	1/K	10.8 · 10 ⁻⁶	
α for aluminum		23 · 10 ⁻⁶	
α for plastic		65 · 10 ⁻⁶	
α for austenitic steel		16 · 10 ⁻⁶	
α for titanium		9 · 10 ⁻⁶	
α for silica		0.5 · 10 ⁻⁶	
Tolerance of temperature response¹⁾	1/K	±0.3 · 10 ⁻⁶	
Adaptation of temperature response in the range²⁾	°C	-10 ... +120	
Maximum elongation³⁾		Absolute strain value ε for positive direction Absolute strain value ε for negative direction	
at reference temperature when using CA80 adhesive			
Strain gage with leads	μm/m	100,000 ±10% 100,000 ±10%	
Fatigue strength³⁾			
at reference temperature when using X60 adhesive			
Achievable number of load cycles L _w with alternating strain ε _w = ±1,000 μm/m			
Strain gage with leads			
Variation of zero point ε _m Δ ≤300 μm/m		>>10 ⁷ (test was aborted at 10 ⁷)	
Variation of zero point ε _m Δ ≤30 μm/m		>10 ⁷ (test was aborted at 10 ⁷)	

Fonte: HBM, 2026. Adaptado pelo autor

O projeto fez uso de quatro pares de pontes de *Wheatstone* completas, e utilizou o modelo de *strain gauge* em V, também conhecido como *strain gauge* Espinha de Peixe. A Figura 5 mostra um exemplo deste modelo de extensômetro. A necessidade de se utilizar este tipo de ferramenta é justificada pois era requisitado que os *strain gauges* coletassem o efeito do cisalhamento no apoio ao aro da cadeira, que se assemelha a uma barra fixa. No projeto foram utilizados os sensores PA-13-125RB-120 da fabricante Excel Sensors (EXCEL, 2026).

A configuração em ponte de *Wheatstone* completa é bastante útil para o uso de strain gauges pois é a que apresenta maior sensibilidade à deformação.

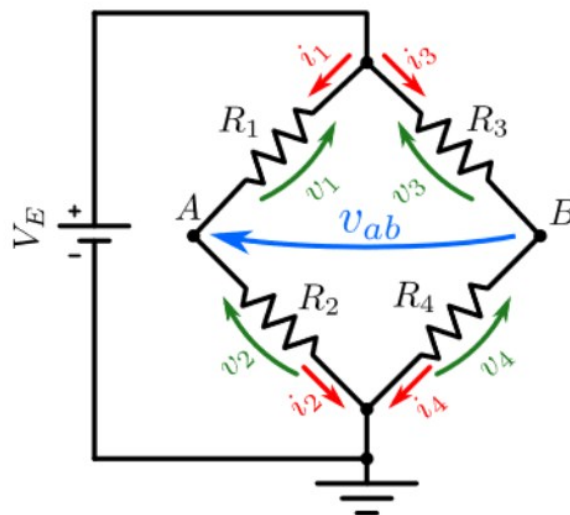
Figura 5 – *Strain gauge* do tipo espinha de peixe



Fonte: EXCEL, 2026

Esse tipo de circuito permite com que seja medida uma variação das tensões nos nós centrais v_{ab} dessa malha, de modo a permitir com que sejam obtidas com bastante sensibilidade dados sobre a deformação do sólido.

Figura 6 – Ponte de Wheatstone



Fonte: SILVA, 2022

É possível dessa forma obter a equação para a aquisição do sinal conforme veremos a seguir, como vista em (SILVA, 2022). Seja dada uma ponte de Wheatstone, considere que cada strain gauge possua valores distintos R_1 , R_2 , R_3 , e R_4 , conforme mostra a Figura 6.

A partir da leis dos nós, é possível deduzir a equação da ponte, que se encontra na seguinte forma:

$$\frac{V_{AB}}{V_E} = \frac{R_2}{R_1 + R_2} - \frac{R_4}{R_3 + R_4} \quad (2)$$

Note que a ponte se encontra em equilíbrio quando temos que $R_1 = R_2 = R_A$ e $R_3 = R_4 = R_B$. Assim, quando ocorrem variações das resistências, essas são dadas pelas equações $R_1 = R_A + \Delta R_1$, $R_2 = R_A + \Delta R_2$, $R_3 = R_B + \Delta R_3$ e $R_4 = R_B + \Delta R_4$.

Substituindo os valores na equação da Ponte de *Wheatstone*, obteremos:

$$\frac{V_{AB}}{V_E} = \frac{R_A + \Delta R_2}{R_A + \Delta R_1 + R_A + \Delta R_2} - \frac{R_B + \Delta R_4}{R_B + \Delta R_3 + R_B + \Delta R_4} \quad (3)$$

Podemos dividir o numerador e denominador de cada termo pelas suas resistências substituídas R_A e R_B , o que nos permite chegar a uma equação com o formato subsequente:

$$\frac{V_{AB}}{V_E} = \frac{1 + R_A + \frac{\Delta R_2}{R_A}}{2 + \frac{\Delta R_1}{R_A} + \frac{\Delta R_2}{R_A}} - \frac{1 + R_B + \frac{\Delta R_4}{R_B}}{2 + \frac{\Delta R_3}{R_B} + \frac{\Delta R_4}{R_B}} \quad (4)$$

Por fim, para o caso de todos os Strain gauges possuírem o mesmo fator G , conseguiremos a expressão $G_{\epsilon_1} = \frac{\Delta R_1}{R_A}$, $G_{\epsilon_2} = \frac{\Delta R_2}{R_A}$, $G_{\epsilon_3} = \frac{\Delta R_3}{R_B}$ e $G_{\epsilon_4} = \frac{\Delta R_4}{R_B}$ para cada strain gauge, e assim podemos substituir na fórmula tendo como finalidade a seguinte expressão:

$$\frac{V_{AB}}{V_E} = \frac{1 + G_{\epsilon_2}}{2 + G_{\epsilon_1} + G_{\epsilon_2}} - \frac{1 + G_{\epsilon_4}}{2 + G_{\epsilon_3} + G_{\epsilon_4}} \quad (5)$$

Realizando as somas e multiplicações para os denominadores:

$$\frac{V_{AB}}{V_E} = \frac{(1 + G_{\varepsilon_2})(2 + G_{\varepsilon_3} + G_{\varepsilon_4}) - (1 + G_{\varepsilon_4})(2 + G_{\varepsilon_1} + G_{\varepsilon_2})}{(4 + 2G_{\varepsilon_3} + 2G_{\varepsilon_4}) + (2G_{\varepsilon_1} + G^2_{\varepsilon_1 \varepsilon_3} + G^2_{\varepsilon_1 \varepsilon_4}) + (2G_{\varepsilon_2} + G^2_{\varepsilon_2 \varepsilon_3} + G^2_{\varepsilon_2 \varepsilon_4})} \quad (6)$$

De acordo com (SILVA, 2022), como $G \approx 2$ para strain gauges metálicos, teremos então $G_{\varepsilon_2} \ll 2G_{\varepsilon_3} \ll 4$ e portanto é possível desprezar esses termos na equação anterior. A equação portanto possui um formato como se segue, considerando apenas o único termo significativo que é 4:

$$\frac{V_{AB}}{V_E} \approx \frac{(1 + G_{\varepsilon_2})(2 + G_{\varepsilon_3} + G_{\varepsilon_4}) - (1 + G_{\varepsilon_4})(2 + G_{\varepsilon_1} + G_{\varepsilon_2})}{4} \quad (7)$$

Finalmente, é possível melhorar ainda mais a expressão com o a expansão seguida da fatoração dos termos presentes no numerador. O resultado do desenvolvimento dos cálculos encontra-se na Equação 8:

$$\frac{V_{AB}}{V_E} \approx \frac{G(-E_1 + E_2 + E_3 - E_4)}{4} \quad (8)$$

Podemos também enunciar a equação da seguinte forma, de modo com que os resistores estejam presentes:

$$\frac{V_{AB}}{V_E} \approx \frac{G(-\frac{\Delta R_1}{R_1} + \frac{\Delta R_2}{R_2} + \frac{\Delta R_3}{R_3} - \frac{\Delta R_4}{R_4})}{4} \quad (9)$$

É importante frisar que todos os strain gauges estão sofrendo uma deformação diferente e positiva, mas note que os strain gauges 2 e 3 causam um aumento na tensão v_{ab} enquanto que os demais gauges 1 e 4 causam sua diminuição (SILVA, 2022).

5. MICROCONTROLADOR – ESP32 – WROOM – 32

O ESP32-WROOM-32 (ESPRESSIF, ESP32-WROOM-32, 2026) é um Microcontroller Unit (MCU) equipado com um processador dual-core de 240 MHz, 4 MB de Flash e 520 KB de SRAM. Ele suporta Wi-Fi de 2,4 GHz (802.11 b/g/n) e Bluetooth 4.2, além de mais de 36 GPIOs, 18 canais ADC e consumo de energia ultrabaixo (5 μ A em modo de hibernação profunda. Estas características o tornam interessante para aplicação neste projeto.

Principais Características Técnicas:

- Processador: CPU dual-core Xtensa LX6 de 32 bits, ajustável entre 80 MHz e 240 MHz.
- Conectividade sem fio: *Wi-Fi* integrado de 2,4 GHz (até 150 Mbps) e *Bluetooth* v4.2 BR/EDR e BLE.
- Memória: 4 MB de flash embutida (padrão), 520 KB de SRAM e 448 KB de ROM.
- Periféricos: 18 canais ADC (12 bits), 2 canais DAC (8 bits), 10 sensores de toque capacitivos, UART, SPI, I2C, I2S, PWM e um sensor Hall integrado.
- Consumo de energia: O modo de hibernação profunda consome apenas 5 μ A; opera a 3,3 V.
- Antena: Normalmente antena PCB integrada (WROOM-32D/32E), com opção de conector U.fl disponível (WROOM-32U).
- Segurança: Segurança completa por hardware, incluindo AES-128/256, inicialização segura e criptografia de flash.

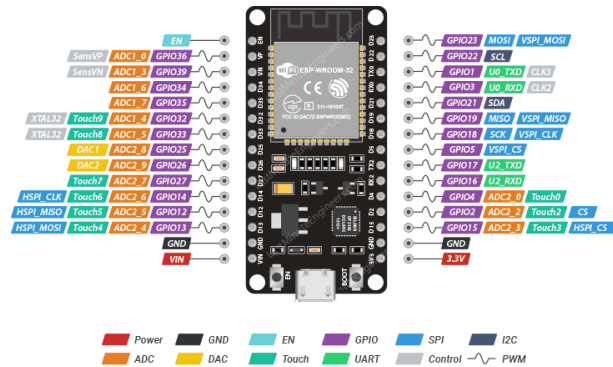
A placa de desenvolvimento ESP32 DevKit V1 contém um MCU ESP32-WROOM-32 e possui um total de 30 pinos (Figura 7). As funções dos pinos são mostradas na Figura 8.

Figura 7 – ESP32 DevKit V1



Fonte: ESPRESSIF, ESP32-WROOM-32

Figura 8 – ESP32 Development Board

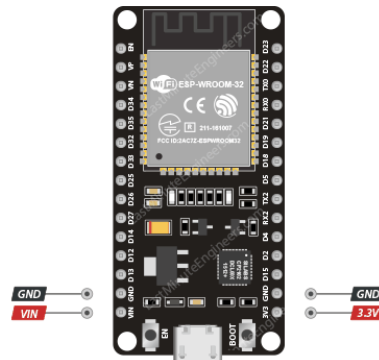


Fonte: Prabhu, 2026

5.1 Pinos de alimentação

Existem dois pinos de alimentação: o pino V_{IN} e o pino 3V3 (Figura X). O pino V_{IN} pode ser usado para alimentar diretamente o ESP32 e seus periféricos através de uma fonte de alimentação regulada de 5V. O pino 3V3 é a saída do regulador de tensão integrado, podendo fornecer até 600mA de corrente. O pino GND é o pino de aterramento (Prabhu, 2026).

Figura 9 – Pinos de alimentação

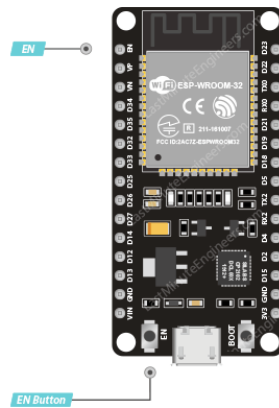


Fonte: Prabhu, 2026

5.2 Reset

O pino EN (Figura 10) é o pino de habilitação do ESP32, que por padrão está em nível alto. Quando em nível alto, o chip é habilitado; quando em nível baixo, o chip é desabilitado (Prabhu, 2026).

Figura 10 – Pino de Reset



Fonte: Prabhu, 2026

5.3 General Purpose Input Output (GPIO)

A placa ESP32 possui 25 pinos GPIO que podem ser configurados com diferentes funções através da programação dos respectivos registradores. Existem vários tipos de GPIOs: somente digitais, com suporte a analógico, com suporte a toque capacitivo etc.

Os GPIOs com suporte a analógico e os com suporte a toque capacitivo podem ser configurados como GPIOs digitais. A maioria desses GPIOs digitais pode ser configurada com resistores de pull-up ou pull-down internos, ou ainda com alta impedância (Figura 11).

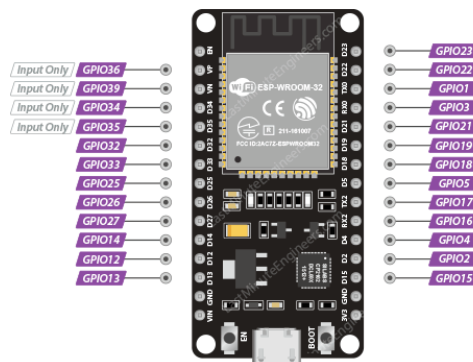
Todos os pinos GPIO podem ser configurados para trabalhar com interrupções. Alguns pinos possuem modos de trabalho específicos que não podem ser alterados como por exemplo os pinos GPIO 34, 35, 36 e 39 que não podem ser configurados como saídas. Podem ser usados como entradas digitais ou analógicas, ou para outras finalidades. Também não possuem resistores internos de pull-up e pull-down, ao contrário dos outros pinos GPIO (Prabhu, 2026).

5.4 Serial Peripheral Interface (SPI)

É um protocolo de dados seriais síncrono usado por microcontroladores para se comunicarem com um ou mais periféricos. Por exemplo, a placa ESP32 se comunicando com um sensor que suporta SPI ou com outro microcontrolador (Prabhu, 2026).

Em uma comunicação SPI, sempre existe um controlador (também chamado de mestre) que controla os dispositivos periféricos (também chamados de escravos).

Figura 11 – Pinos GPIO



Fonte: Prabhu, 2026

Os dados podem ser enviados e recebidos simultaneamente. Isso significa que o mestre pode enviar dados para um escravo e um escravo pode enviar dados para o mestre ao mesmo tempo.

Para a comunicação SPI são necessárias quatro conexões:

- **MISO:** Master In Slave Out
- **MOSI:** Master Out Slave In
- **SCK:** Serial Clock
- **CS /SS:** Chip Select (usado para selecionar o dispositivo quando múltiplos periféricos são usados no mesmo bus SPI).

O ESP32 (Figura 12) possui três interfaces SPI (SPI, HSPI e VSPI) nos modos escravo e mestre. Essas interfaces SPI também suportam os recursos SPI de uso geral listados abaixo:

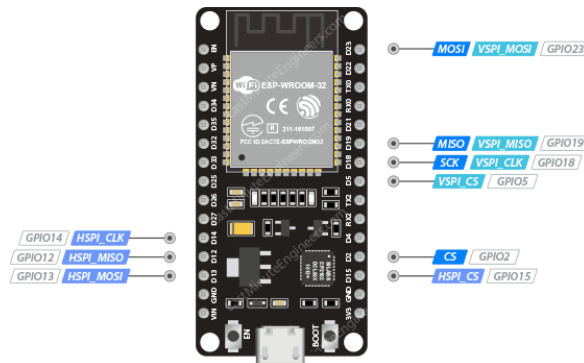
- 4 modos de temporização para transferência no formato SPI
- Até 80 MHz e clocks divididos de 80 MHz
- FIFO de até 64 bytes

Apenas as interfaces VSPI e HSPI são utilizáveis, e o terceiro barramento SPI é usado pelo chip de memória flash integrado. Os pinos VSPI são comumente usados em bibliotecas padrão.

5.5 Analog to Digital Converter (ADC)

A placa ESP32 DevKit V1 tem 15 pinos que contam com ADC (Figura 13). É um ADC de 12 bits, o que significa que ele pode detectar 4096 (2^{12}) níveis analógicos discretos.

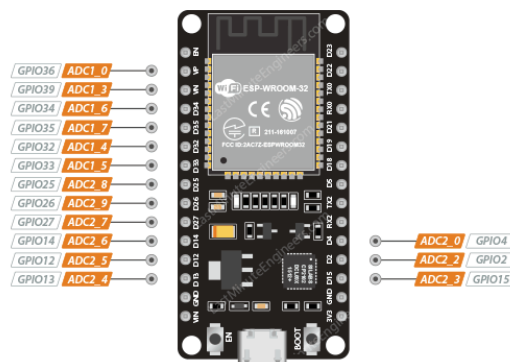
Figura 12 – Pinos SPI



Fonte: Prabhu, 2026

Ele converte tensões de entrada que variam de 0 a 3,3 V (tensão de operação) em valores inteiros que variam de 0 a 4095. Isso resulta em uma resolução de 3,3 volts/4096 unidades, ou 0,0008 volts (0,8mV) por unidade. A resolução (9, 10, 11 e 12 bits) pode ser configurada por software. Os pinos ADC2 não podem ser usados quando o Wi-Fi estiver ativado. Caso seja necessário o uso de Wi-Fi, deve-se usar os pinos do ADC1.

Figura 13 – Pinos ADC

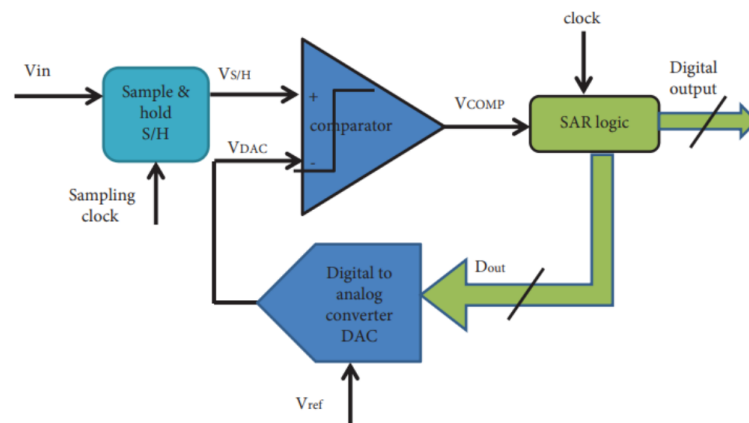


Fonte: Prabhu, 2026

O ESP32 utiliza para conversão ADC um conversor de aproximação sucessiva do tipo. O conversor de aproximação sucessiva é um sistema de realimentação que faz uso da tentativa e erro para estimar a entrada analógica de um sinal e m um código digital correspondente. Esse sistema é formado por um registrador de aproximações sucessivas e analógico-digital que funciona com o uso de um registrador de aproximações sucessivas (tipo Successive Approximation Register – SAR) e um conversor A/D em realimentação com um comparador de tensão, como visto na Figura 14 (GREBENE, 2002; ARAFA et al., 2023).

O SAR faz uso de um switch denominado Sample-and-Hold (S/H), onde o sinal é primeiramente amostrado, enquanto o conversor D/A do circuito converte um código digital guardado no registrador em um valor analógico. É feita então uma comparação através do circuito comparador, que verifica a entrada do switch S/H e a do conversor D/A o que produz uma saída digital que instrui a lógica a atualizar o registrador que contém a lógica do SAR (ARAFA et al., 2023). A lógica SAR em maiores detalhes é determinada de forma sucessiva. Existem um registrador de N-bits e um registrador de retenção. Ambos são limpos antes do processo de aquisição do sinal analógico acontecer.

Figura 14 – Esquema de um ADC SAR



Fonte: ARAFA et al., 2023

Durante a etapa de conversão, é inserido um bit de valor 1 como bit de prova para o MSB no registrador de retenção, com o restante dos seus bits contendo o valor 0. Caso o valor analógico resultante do conversor D/A ser menor ou igual ao seu valor de saída, o estado do comparador se mantém inalterado, e o valor 1 continua para o MSB. Caso contrário esse valor é repostado por 0. Assim, no próximo ciclo, o valor 1 é colocado como bit de prova para o segundo bit mais significativo, seguindo a mesma lógica anterior até que sejam realizados N ciclos sucessivamente (GREBENE, 2002; ARAFA et al., 2023).

De acordo com (GREBENE, 2002), este é um dos conversores mais usados, dado que possui uma resolução relativamente alta e uma alta velocidade. No entanto, as incertezas nos bits dado ao ruído do comparador, fonte ou tempo de assentamento do circuito trazem desvantagens ao uso desse tipo de conversor A/D (ARAFA et al., 2023).

O ADC precisa de uma tensão de referência para comparar com a tensão de entrada, a fim de calcular os valores digitais (variando de 0 a 4095, já que é um ADC de 12 bits) da tensão de entrada. Essa tensão de referência, chamada V_{ref} (Voltage Reference), ajuda o ADC a mapear as tensões de entrada para essa faixa. O ESP32 usa uma V_{ref} de aproximadamente 1,1 V. Isso significa que ele só pode mapear tensões de entrada entre 0 V e 1,1 V. Para valores acima de 1,1V deve ser realizada a atenuação do sinal de entrada. Dessa forma, o ADC pode mapear tensões maiores que 1,1 V para a faixa de 0 a 4095.

O ESP32 suporta quatro níveis de atenuação, com faixas de linearidade recomendadas para melhor precisão (MORAIS, 2026):

- Atenuação 0 dB (0 – 1.1V): melhor precisão linear abaixo de 950mV.
- Atenuação 2.5 dB (0 – 1.5V): boa linearidade até aproximadamente 1050mV.
- Atenuação 6 dB (0 – 2.2V): linearidade razoável até 1300mV.
- Atenuação 11 dB (0 – 3.3V): faixa completa, mas com linearidade ruim acima de 2.6V

Cada ESP32 pode ter até 6% de diferença nas leituras, boa parte devido à tensão de referência (V_{ref}) do ADC haver grande variação (1000 – 1200 mV). Essa variação na V_{ref} ocasiona leituras diferentes entre os chips (MORAIS, 2026).

A calibração do ADC minimiza esses efeitos ao ajustar as leituras com base em parâmetros armazenados na *eFuse* de cada chip durante a fabricação. Esses parâmetros contêm informações sobre a tensão de referência real e características de linearidade específicas daquele chip, permitindo que o driver de calibração compense as variações e forneça resultados mais precisos (MORAIS, 2026).

Durante a fabricação (ESPRESSIF, ADC calibration driver, 2026), cada chip ESP32 passa por um processo de caracterização onde parâmetros específicos são medidos e gravados na *eFuse*. Esses parâmetros incluem:

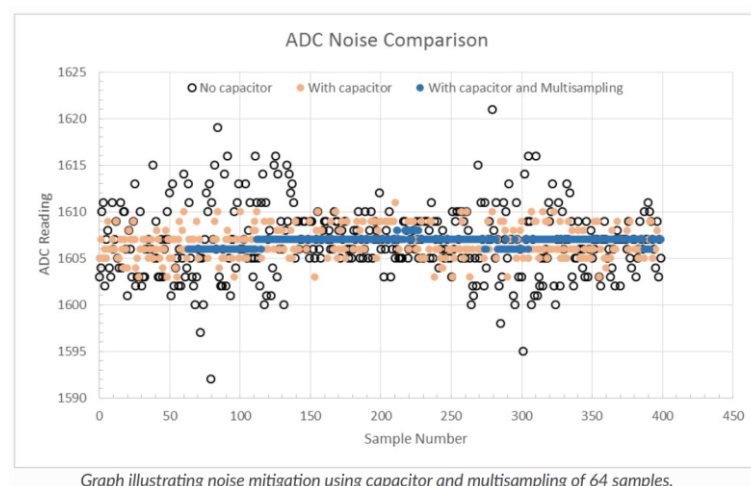
- Tensão de referência real do ADC (que pode variar entre 1000–1200 mV)
- Coeficientes de calibração para diferentes níveis de atenuação
- Parâmetros de linearidade para compensar a não-linearidade do ADC

O driver de calibração lê esses valores da *eFuse* e os utiliza para criar uma função de conversão que transforma o valor bruto (raw) do ADC em uma tensão calibrada em milivolts.

Práticas que podem melhorar a qualidade das medições (ESPRESSIF, ADC calibration driver, 2026):

- *Oversampling*: realizar múltiplas leituras e calcular a média reduz drasticamente o ruído. O número ideal de amostras depende da aplicação, mas valores entre 32 e 128 são comuns;
- Capacitores de *by-pass*: adicionar um capacitor cerâmico de 100 nF entre o pino do ADC e o GND pode melhorar significativamente a estabilidade das leituras;
- Atenuação Adequada: o ESP32 oferece diferentes níveis de atenuação que determinam a faixa de tensão que pode ser medida, conforme visto anteriormente;
- ADC2 com Wi-Fi Ativo: o ADC2 não pode ser usado quando o Wi-Fi está ativo no ESP32;
- Tipo de Calibração: verificar o tipo de calibração foi utilizado (*eFuse*, *Two Point*, *eFuse*, V_{ref} ou valor padrão).

Figura 15 – Atenuação de ruídos no ADC



Fonte: ESPRESSIF, ADC calibration driver, 2026

6. Módulo transceiver nRF24L01+

O ESP32 apresenta conectividade sem fio através da utilização de *WiFi*, *Bluetooth* e *Bluetooth Low Energy* (BLE) que são alternativas que podem ser utilizadas no projeto para a transmissão de dados. Porém, deve-se levar em consideração qual o consumo de energia em cada um destas modalidades, tanto na transmissão quanto na recepção, uma vez que os circuito de condicionamento e transmissão de sinais serão alimentados por pilhas recarregáveis de lítio.

O consumo de energia no Modo Ativo varia bastante e é determinado principalmente pela atividade do módulo de RF, que é a parte do ESP32 que mais consome energia.

O Modo Ativo é o estado padrão do ESP32, onde o chip opera em sua capacidade operacional máxima. Nesse modo, todos os periféricos digitais estão ativos, os clocks da CPU estão funcionando e o módulo de RF, responsável pelas comunicações Wi-Fi e Bluetooth, é totalmente capaz de transmitir e receber dados. Como muitos componentes são ligados simultaneamente, o ESP32 consome mais energia nesse modo.

De acordo com dados fornecidos pelo fabricante (ESPRESSIF. ESP32–WROOM–32, 2026) durante a transmissão Wi-Fi, o consumo máximo de corrente pode variar entre 180 mA e 240 mA , dependendo do esquema de modulação (o método usado para codificar os dados), do nível de potência de transmissão (potência TX) e da taxa de dados utilizada.

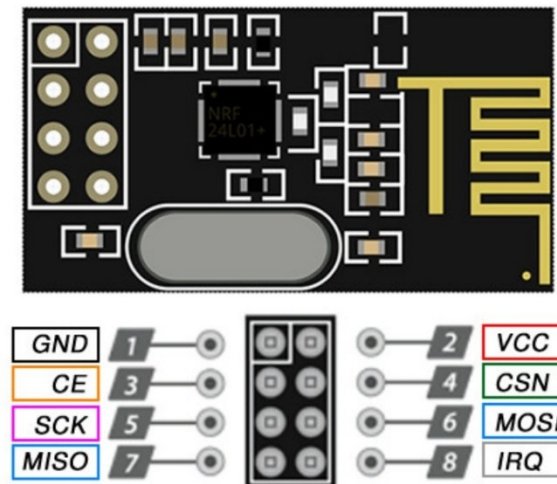
O modo de recepção Wi-Fi é menos exigente do que o de transmissão, mas ainda requer uma quantidade significativa de energia. Durante a recepção Wi-Fi, o dispositivo consome entre 95 mA e 100 mA .

As operações Bluetooth Classic e Bluetooth Low Energy (BLE) geralmente consomem menos energia em comparação com o Wi-Fi. Ao transmitir pacotes Bluetooth a 0 dBm, o chip normalmente consome cerca de 130 mA e, durante a recepção Bluetooth, consome entre 95 mA e 100 mA .

Dado o consumo do ESP32 nestas modalidades de transmissão sem fio, torna-se interessante a utilização de módulos que permitam transmissão de dados sem fio e que apresentem um menor consumo de energia no modo ativo e também nos modos de hibernação.

O módulo nRF24L01+ (SPARKFUN, 2026) foi utilizado no projeto por reunir estas características. É um transceptor de 2,4 GHz em chip único com um mecanismo de protocolo de banda base integrado (*Enhanced ShockBurst*), adequado para aplicações sem fio de ultrabaixo consumo de energia. O nRF24L01+ foi projetado para operação na banda de frequência ISM mundial de 2.400 a 2.4835 GHz.

Figura 16 – Módulo transceiver nRF24L01+



Fonte: SPARKFUN, 2026

A configuração e operação do nRF24L01+ é realizada por meio de uma Interface Periférica Serial (SPI). O mapa de registros, acessível pela SPI, contém todos os registros de configuração do nRF24L01+ e é acessível em todos os modos de operação do chip.

O mecanismo de protocolo de banda base integrado (*Enhanced ShockBurst*) é baseado em comunicação por pacotes e suporta vários modos, desde operação manual até operação autônoma avançada de protocolo. FIFOs internos garantem um fluxo de dados suave entre o front-end de rádio e o MCU do sistema.

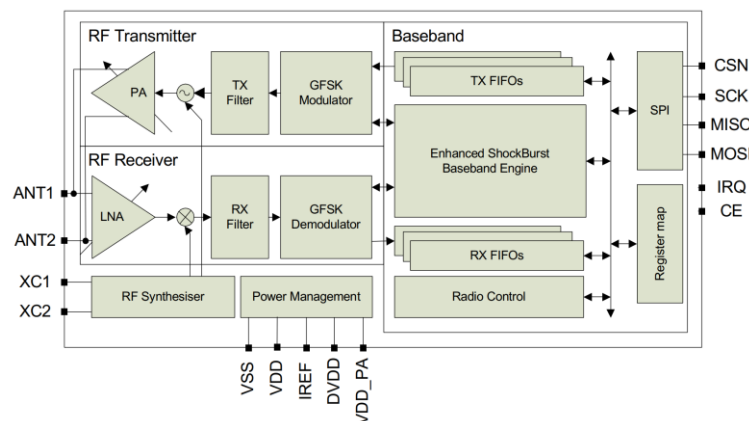
O front-end de rádio usa modulação GFSK. Possui parâmetros configuráveis por *software*, como canal de frequência, potência de saída e taxa de dados sem fio. O nRF24L01+ suporta taxas de dados sem fio de 250 kbps, 1 Mbps e 2 Mbps. A alta taxa de dados sem fio, combinada com dois modos de economia de energia, torna o nRF24L01+ adequado para projetos de ultrabaixo consumo de energia.

Características do nRF24L01+:

- Operação mundial na banda ISM de 2.4 GHz;
- 126 canais de RF;
- Interface comum de RX e TX;
- Modulação GFSK;
- Taxa de dados de 250 kbps, 1 Mbps e 2 Mbps;
- Potência de saída programável: 0, -6, -12 ou -18 dBm;

- Consumo de 11,3 mA com potência de saída de 0 dBm;
- AGC rápido para faixa dinâmica aprimorada;
- Filtros de canal integrados;
- Consumo de 13,5 mA a 2 Mbps;
- Sensibilidade de -82 dBm a 2 Mbps;
- Sensibilidade de -85 dBm a 1 Mbps;
- Sensibilidade de -94 dBm a 250 kbps;
- Payload dinâmico de 1 a 32 bytes;
- Manipulação automática de pacotes;
- Manipulação automática de transações de pacotes;
- Multireceptor de 6 canais de dados para redes em estrela 1: 6;
- Regulador de tensão integrado;
- Faixa de alimentação de 1,9 a 3,6V;
- Modos ociosos com tempos de inicialização rápidos para gerenciamento avançado de energia;
- Modo *Standby* de 26μA, modo de desligamento de 900nA;
- Inicialização máxima de 1,5ms a partir do modo de desligamento;
- Inicialização máxima de 130μs a partir do modo *Standby*;
- SPI de hardware de 4 pinos, máximo de 10Mbps;
- 3 FIFOs TX e RX separados de 32 bytes;
- Entradas com tolerância de 5V.

Figura 17 – Diagrama de blocos do nRF24L01+



Fonte: SPARKFUN, 2026

6.1 Enhanced ShockBurst™

O *Enhanced ShockBurst* (ESB) (SPARKFUN, 2026) é uma camada de enlace de dados baseada em pacotes que apresenta montagem e temporização automática de pacotes, reconhecimento automático e de transmissões de pacotes. O ESB permite a implementação de comunicação de ultrabaixo consumo e alto desempenho com microcontroladores host de baixo custo. Os recursos do ESB possibilitam melhorias significativas na eficiência energética para sistemas bidirecionais e unidirecionais, sem adicionar complexidade ao controlador host.

Principais recursos:

- Comprimento dinâmico da carga útil de 1 a 32 bytes;
- Manipulação automática de pacotes;
- Manipulação automática de transações de pacotes;
- Reconhecimento automático com carga útil;
- Retransmissão automática;
- Multireceptor de 6 canais de dados para redes em estrela 1: 6.

O ESB para manipulação e temporização automática de pacotes. Durante a transmissão, o ESB monta o pacote e sincroniza os bits no pacote de dados para transmissão. Durante a recepção, o ESB busca constantemente um endereço válido no sinal demodulado.

Quando encontra um endereço válido, processa o restante do pacote e o valida por CRC. Se o pacote for válido, a carga útil é movida para um slot vago nos FIFOs de recepção. Todo o processamento de bits em alta velocidade e o sincronismo são controlados pelo ESB.

O ESB aprimorado oferece tratamento automático de transações de pacotes para facilitar a implementação de um enlace de dados bidirecional confiável. Uma transação de pacote do ESB aprimorado é uma troca de pacotes entre dois transceptores, com um transceptor atuando como Receptor Primário (PRX) e o outro como Transmissor Primário (PTX). Uma transação de pacote do ESB aprimorado é sempre iniciada por uma transmissão de pacote do PTX e é concluída quando o PTX recebe um pacote de confirmação (pacote ACK) do PRX. O PRX pode anexar dados do usuário ao pacote ACK, permitindo um enlace de dados bidirecional.

O processamento automático de transações de pacotes funciona da seguinte forma:

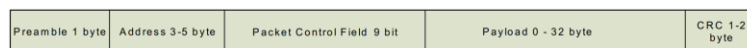
- Inicia a transação transmitindo um pacote de dados do PTX para o PRX. O ESB configura automaticamente o PTX no modo de recepção para aguardar o pacote ACK;

- Se o pacote for recebido pelo PRX, o ESB monta e transmite automaticamente um pacote de confirmação (pacote ACK) para o PTX antes de retornar ao modo de recepção;
- Se o PTX não receber o pacote ACK imediatamente, o ESB retransmite automaticamente o pacote de dados original após um atraso programável e configura o PTX no modo de recepção para aguardar o pacote ACK.

No ESB, é possível configurar parâmetros como o número máximo de retransmissões e o atraso entre uma transmissão e a próxima retransmissão. Todo o processamento automático é feito sem a intervenção do MCU.

O formato do pacote do ESB é mostrado na Figura 18.

Figura 18 – Formato do pacote do nRF24L01+



Fonte: SPARKFUN, 2026

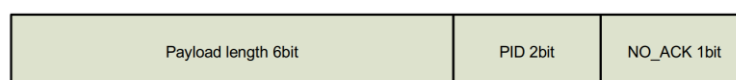
O preâmbulo é uma sequência de bits usada para sincronizar o demodulador do receptor com o fluxo de bits de entrada. O preâmbulo tem um byte de comprimento e é 01010101 ou 10101010.

Se o primeiro bit do endereço for 1, o preâmbulo é automaticamente definido como 10101010 e, se o primeiro bit for 0, o preâmbulo é automaticamente definido como 01010101. Isso é feito para garantir que haja transições suficientes no preâmbulo para estabilizar o receptor.

O endereço garante que o pacote seja detectado e recebido pelo receptor correto, evitando interferências acidentais entre vários sistemas nRF24L01+.

A Figura 19 mostra o formato do campo de controle do pacote de 9 bits, com o bit mais significativo (MSB) à esquerda.

Figura 19 – Campo de 9 bits de controle do pacote



Fonte: SPARKFUN, 2026

Este campo de 6 bits especifica o comprimento da carga útil em bytes. O comprimento da carga útil pode ser de 0 a 32 bytes. Codificação: 000000 = 0 byte (usado apenas em pacotes ACK vazios). Este campo é usado apenas se a função Dynamic Payload Length estiver habilitada.

O campo PID (*Packet identification*) de 2 bits é usado para detectar se o pacote recebido é novo ou retransmitido.

O *Payload* é o conteúdo do pacote definido pelo usuário. Ela pode ter de 0 a 32 bytes de largura e é transmitida pelo ar quando é carregada no nRF24L01+.

O ESB oferece duas alternativas para lidar com o comprimento da carga útil: estática e dinâmica.

O padrão é o comprimento estático da carga útil. Com o comprimento estático da carga útil, todos os pacotes entre um transmissor e um receptor têm o mesmo comprimento. O comprimento estático da carga útil é definido pelos registradores RX_PW_Px no lado do receptor.

O comprimento da carga útil no lado do transmissor é definido pelo número de bytes enviados para o TX_FIFO e deve ser igual ao valor no registrador RX_PW_Px no lado do receptor.

O CRC (Cyclic Redundancy Check) é o mecanismo obrigatório de detecção de erros no pacote. Ele tem 1 ou 2 bytes e é calculado com base no endereço, no Campo de Controle do Pacote e no *Payload*.

7. METODOLOGIA

7.1 Circuito de condicionamento de sinal e transmissão de dados

O circuito de condicionamento de sinal e transmissão de dados será dividido em três partes para melhor entendimento e detalhados em sequência.

7.1.1 Terra virtual com amplificador operacional

O componente principal do circuito de condicionamento é o amplificador de instrumentação INA 128, que necessita para a sua correta operação, alimentação por fonte simétrica. A utilização de uma fonte simétrica em amplificadores de instrumentação é crucial para garantir a fidelidade e a integridade da medição de sinais, especialmente quando estes são pequenos ou oscilam em torno de zero (sinais CA ou contínuos que podem inverter a polaridade).

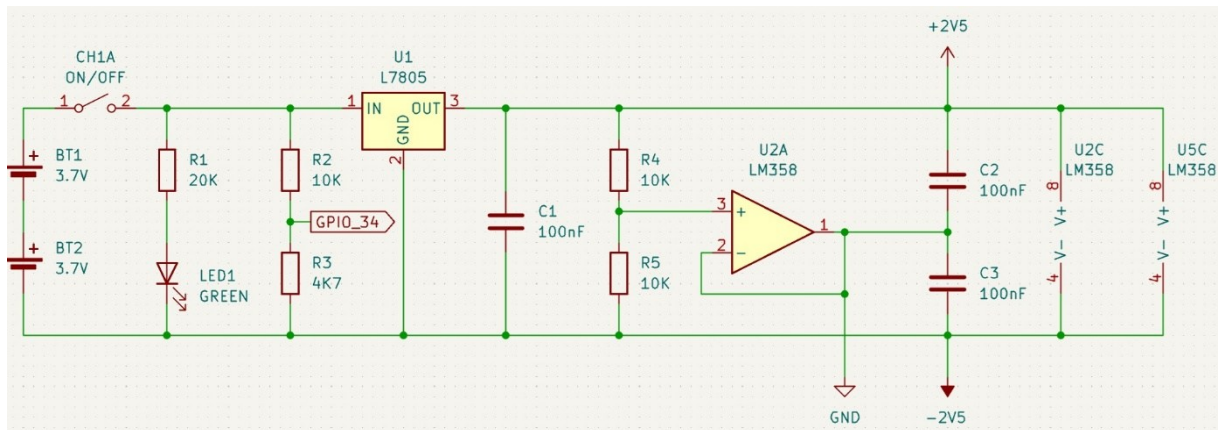
Os principais motivos para essa utilização são:

- **Amplificação de Sinais Bidirecionais (CA):** Permite que o amplificador processe sinais que alternam entre positivo e negativo (corrente alternada) sem ceifar (cortar) a parte negativa do sinal;
- **Saída Referenciada ao Terra (Offset Nulo):** Com fonte simétrica, a saída do amplificador pode chegar a 0V com precisão quando as entradas são iguais, o que é essencial para aplicações de medição e instrumentação;
- **Polarização Adequada dos AmpOps Internos:** Amplificadores de instrumentação são baseados em amplificadores operacionais internos que funcionam de forma mais linear e estável com alimentação simétrica;
- **Ideal para Pequenos Sinais:** Fundamental em circuitos que exigem alto ganho e alta rejeição de modo comum (CMRR), pois mantém a linearidade necessária para não introduzir ruídos no sinal amplificado.

O circuito será alimentado por duas pilhas de lítio do tipo 18650, cuja tensão nominal individual é de 3.7V, totalizando 7.4V. Nesta configuração há apenas um polo positivo e outro negativo da associação em série, sem a presença de um terra.

O uso de um amplificador operacional como seguidor tensão replica a tensão presente na entrada não inversora, permitindo construir uma fonte simétrica do tipo terra virtual com dois resistores de igual valor na entrada não inversora, dividindo por dois a tensão de alimentação. O circuito da Figura 20 mostra o circuito proposto. Pelo datasheet (TEXAS INSTRUMENTS, 2026) o LM358 é capaz de fornecer até 20mA de corrente de saída.

Figura 20 – Circuito de condicionamento: terra virtual



Fonte: Elaborado pelo autor

7.1.2 Amplificador de instrumentação

O amplificador de instrumentação é um circuito diferencial de alta precisão projetado para amplificar sinais analógicos fracos na presença de alto ruído, ideal para sensores. Suas principais características incluem impedância de entrada muito alta, altíssima taxa de rejeição de modo comum ($CMRR > 100$ dB) e ganho ajustável com precisão.

Principais Características e Vantagens:

- Alta Rejeição de Modo Comum (CMRR): essencial para eliminar ruídos, como interferências da rede elétrica, que aparecem da mesma forma em ambas as entradas;
- Alta Impedância de Entrada: minimiza o efeito de carga, garantindo que o sensor não seja carregado e a medição seja fiel ao original.
- Ganho de Tensão Preciso: o ganho é tipicamente definido por um único resistor externo), facilitando o ajuste de ganhos altos;
- Baixo Desvio (*Drift*) e Ruído: Otimizado para sinais CC (corrente contínua) de baixo nível e alta estabilidade térmica.

- Estrutura de 3 Amplificadores Operacionais: geralmente composto por um estágio de entrada com dois amplificadores não-inversores (buffer) seguido por um amplificador subtrator, garantindo linearidade e isolamento.

O amplificador de instrumentação utilizado foi o INA128 (BURR BROWN, 2026), que apresenta um ganho típica da ordem de 120 dB, bastante adequado para amplificar os sinais elétricos de diminuta amplitude provenientes das pontes de *Wheatstone* decorrentes das deformações aplicadas nos *strain gauges*.

O ganho de amplificação do INA128 é dado pela seguinte equação:

$$A = 1 + \frac{50K\Omega}{R_g} \quad (10)$$

O amplificador de instrumentação será alimentado por uma tensão de $\pm 2,5V$ proveniente do circuito da Figura 20. Como o ESP32-WROOM-32 é alimentado com uma tensão de 3,3V, há a necessidade de ser promover um deslocamento da tensão de saída do amplificador de instrumentação para uma tensão na faixa de 0 a 3,3V, que será aplicada em pino do MCU a fim de sofrer conversão pelo ADC do MCU. Para efetuar esse deslocamento, foi utilizado o pino Ref, onde pode ser aplicada uma tensão de $\pm 2,5V$ afim de promover o offset da tensão de saída. O *datasheet* (BURR BROWN, 2026) informa a necessidade da utilização de um amplificador operacional para garantir uma baixa impedância neste pino e para garantir a alta CMMR.

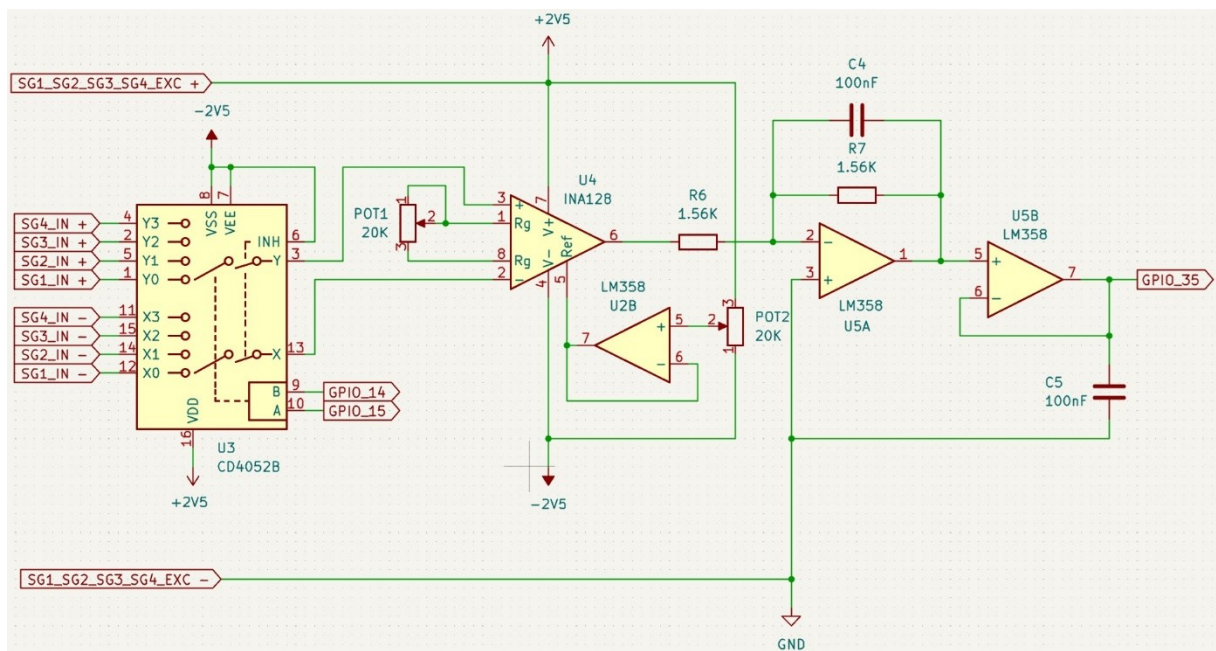
Conforme dados disponíveis no trabalho de COQUEIRO, 2024, é necessário a presença de um filtro passa baixa com frequência de corte de 1KHz a fim de eliminar ruídos indesejados. Para isto foi construído um filtro ativo do tipo *Butterworth* de 1º ordem, um tipo de filtro que não apresenta ondulações (*ripple*) na banda de passagem nem na banda de rejeição, cuja amplitude do sinal diminui suavemente à medida que a frequência se afasta da frequência de corte, que apresenta transição suave e uma resposta de fase relativamente linear, o que resulta em menos distorção de fase em comparação com outros. A frequência de corte calculada foi de:

$$F_c = \frac{1}{2\pi RC} = \frac{1}{2\pi \cdot 1,56K \cdot 100nF} = 1,02KHz \quad (11)$$

Foi incluído um amplificador operacional funcionando como buffer, garantindo uma baixa impedância na saída do filtro *Butterworth*. Na saída foi inserido um capacitor de 100nF a fim de reduzir ruídos na entrada do conversor ADC do MCU conform recomendação do fabricante.

A cadeira de rodas tem quatro *straing gauges* instalados, cujos sinais precisam ser captados e condicionados separadamente. Uma alternativa possível seria o uso de quatro amplificadores de instrumentação e de quatro filtros *Butterworth*, o que aumentaria a complexidade e o custo para a implementação do circuito. Para contornar esta complexidade, foi utilizado o circuito integrado CD4052 (TEXAS INSTRUMENTS, 2026) composto por dois circuitos de multiplexação de quatro entradas, capas de trabalhar tanto com sinais digitais como com sinais analógicos e cujo chaveamento é feito pelo MCU. O circuito proposto é mostrado a Figura 21.

Figura 21 – Circuito de condicionamento: amplificador de instrumentação



Fonte: Elaborado pelo autor

7.1.3 Conversão analógica digital e transmissão dos dados

Após o condicionamento do sinal obtido dos quatro *strain gauges*, este é aplicado no ADC1, canal 7 (GPIO 35). O sinal sofre então uma atenuação de 11dB no ADC, o que permite a leitura de valores entre 0 e 3,3V, com melhor precisão para valores entre 150mV a 2.450mV conforme mencionado em seção anterior

A amostragem é feita a uma taxa de 128.000 amostras por segundo e como a transmissão é feita a uma taxa de 1KHz, é realizado um *oversampling* de 128 amostras por milissegundo. O *oversampling* é necessário para minimizar os erros das medidas realizadas pelo ADC do ESP32 e é recomendado pelo fabricante.

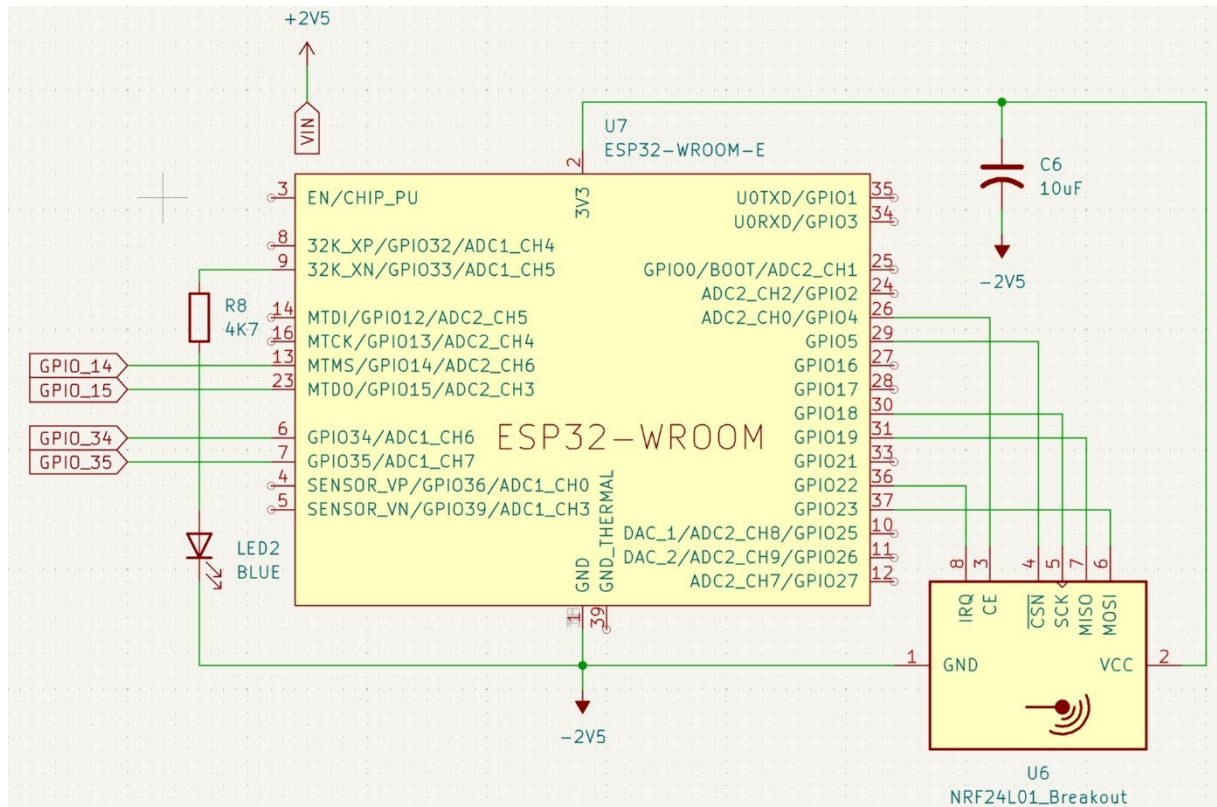
Para se obter altas taxas de amostragem, é necessário do uso de DMA (Direct Memory Access). O DMA no ESP32 permite transferências de dados de alta velocidade entre periféricos (I2S, SPI, UART) e memória sem sobrecarregar a CPU. O uso de buffers DMA, geralmente configurados via `dma_buf_count` e `dma_buf_len` (no I2S, por exemplo), gerencia o balanço entre latência e carga da CPU, alocando blocos de memória interna. Isso é ideal para áudio ou sensores rápidos, suportando taxas de amostragem de 20 kHz a 2 MHz, usando principalmente o ADC1 (ADC2 não suporta DMA no ESP32 clássico).

Há de considerar o fato de que buffers menores diminuem a latência (tempo de resposta), mas aumentam a carga da CPU, enquanto buffers maiores dão mais tempo para o processamento, consumindo mais RAM.

É feita uma média das 128 amostras lidas a cada milissegundo, de cada um dos quatro *strain gauges* e estes valores são transmitidos via módulo nRF24L01+ a uma taxa de 1.000 transmissões a cada segundo.

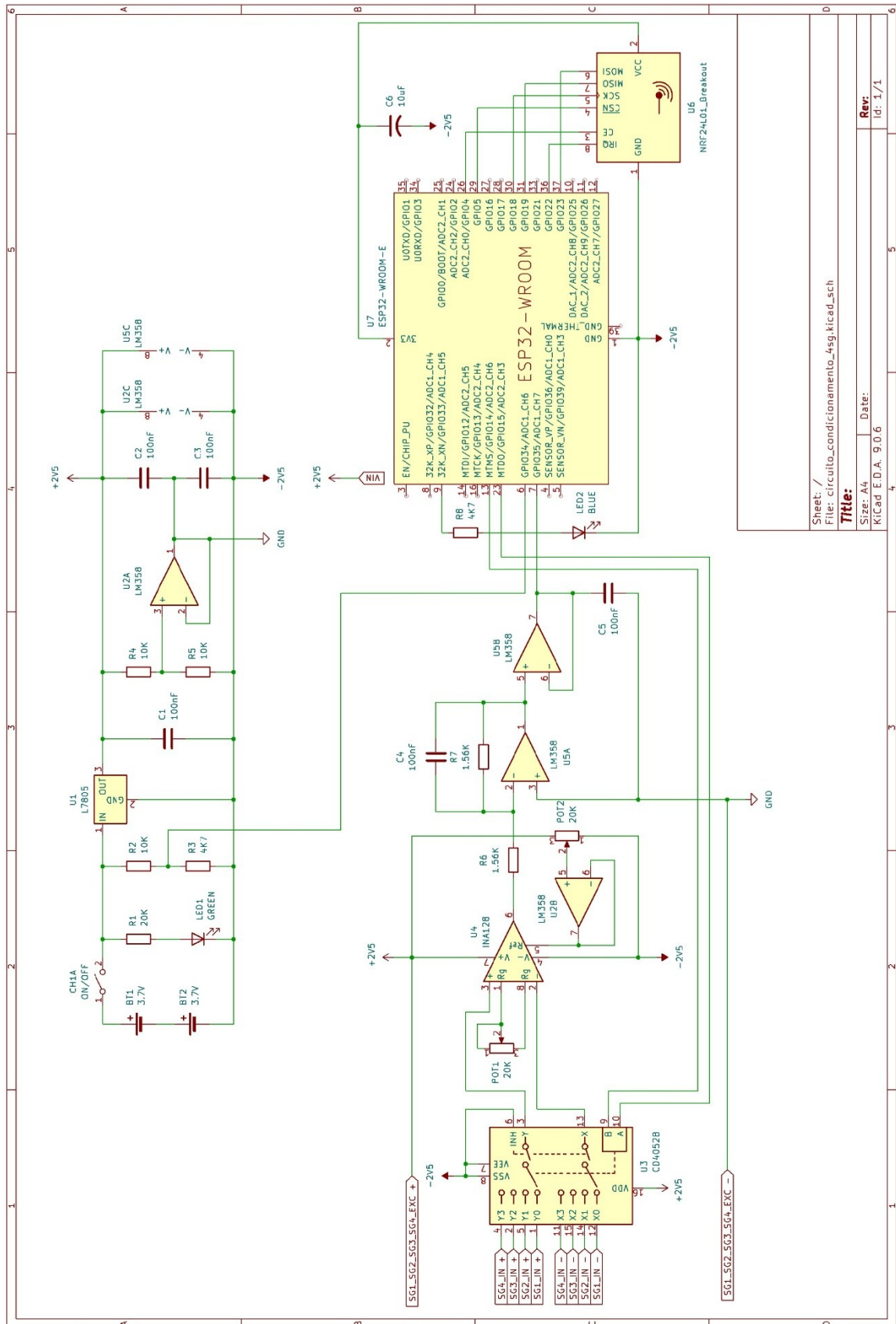
A Figura 22 mostra os componentes desta parte do circuito. A Figura 23 mostra o circuito de condicionamento e transmissão de dados com suas partes integradas.

Figura 22 – Conversão analógica digital e transmissão dos dados



Fonte: Elaborado pelo autor

Figura 23 – Circuito de condicionamento completo



Sheet: /
 File: circuito_condicionamento_4sg.kicad_sch
Title:
 Size: A4
 Date:
 KiCad E.D.A. 9.0.6
Rev:
 Id: 1/1

Fonte: Elaborado pelo autor

8. RESULTADOS E DISCUSSÃO

9. CONCLUSÃO

8. CONCLUSÃO

O presente trabalho apresenta os seguintes objetivos...

REFERÊNCIAS

ARAFA, K. I. et al. **Successive approximation register analog-to-digital converter (sar adc)** for biomedical applications. *Active and Passive Electronic Components*, Hindawi, v. 2023, 2023.

BURR BROWN. **INA128**, 2026. Disponível em: <<https://www.ti.com/lit/ds/symlink/ln358.pdf>>. Acesso em: 18 mar. 2026.

CHETTRI, Lalit; BERA Rabindranath. **A Comprehensive Survey on Internet of Things (IoT) Toward 5G Wireless Systems**. *IEEE Internet Of Things Journal*, v. 7, n. 1, jan. 2020.

COOK, A. M., HUSSEY, S. M., **Assistive technologies: principles and practices**. St. Louis: Mosby, 2015.

COOPER, R. A et al. **Engineering better wheelchairs to enhance community participation**. *IEEE Transactions on Neural Systems and Rehabilitation Engineering*, 14(4), 438–455, 2006. Disponível em: <<http://dx.doi.org/10.1109/TNSRE.2006.888382>>. Acesso em: 10 mar. 2026.

COOPER, R. A.; COOPER, R.; BONINGER, M. L. **Trends and issues in wheelchair technologies**. *Assistive Technology*, Taylor & Francis, v. 20, n. 2, p. 61–72, 2008.

COOPER, R. A. et al. **Evaluation of a pushrim-activated, power-assisted wheelchair**. *Archives of Physical Medicine and Rehabilitation*, Elsevier, v. 82, n. 5, p. 702–708, 2001.

COQUEIRO, I. C. L. **Desenvolvimento e Implementação de Sistema de Condicionamento de Sinal de Força com Comunicação Sem Fio para Cadeira de Rodas Servo Assistida**; 2024; Trabalho de Conclusão de Curso; (Graduação em Engenharia Biomédica) – Universidade Federal do ABC.

CUERVA, V. **Controle da locomoção assistida de cadeiras de rodas manuais por meio do controle de impedância: análise via controle ótimo**. Centro Universitário FEI, São Bernardo do Campo, 2017.

ESPRESSIF. **ADC calibration driver**, 2026. Disponível em: <https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/peripherals/adc/adc_calibration.html>. Acesso em: 10 mar. 2026.

ESPRESSIF. **ESP32-WROOM-32**. Espressif, 2026. Disponível em: <https://documentation.espressif.com/esp32-wroom-32_datasheet_en.pdf>. Acesso em: 10 mar. 2026.

EXCEL. **MODELOS DUPLOS TIPO ESPINHA DE PEIXE**. Excel, 2026. Disponível em: <<https://excelsensor.com.br/extensometros-colaveis-de-resistencia-eletrica/modelos/duplos-espinha-de-peixe/>>. Acesso em: 10 mar. 2026.

HBM. **LD20: Strain gauge para grandes deformações ou compressões, até 10 %**. Disponível em: <<https://www.hbm.com/pt/3451/ld20-strain-gauge-para-grandes>>

[deformacoes-ou-compressoes/?product_type_no=LD20:%20Strain%20gauge%20para%20grandes%20deformacoes%20ou%20compressoes](https://www.deformacoes-ou-compressoes/?product_type_no=LD20:%20Strain%20gauge%20para%20grandes%20deformacoes%20ou%20compressoes). Acesso em: 18 mar. 2026.

HETZ, S. P., LATIMER, A. E., GINIS, K. A. M. **Activities of daily living performed by individuals with SCI: relationships with physical fitness and leisure time physical activity.** Spinal Cord, 47(7), 550–554, 2008. Disponível em: <<http://dx.doi.org/10.1038/sc.2008.160>>. Acesso em: 30 jan. 2026.

IBGE – Instituto Brasileiro de Geografia e Estatística – **População residente por tipo de deficiência permanente**. Censo Demográfico, 2022. Disponível em: <https://agenciadenoticias.ibge.gov.br/agencia-noticias/2012-agencia-de-noticias/noticias/43463-censo-2022-brasil-tem-14-4-milhoes-de-pessoas-com-deficiencia#:~:text=Entre%20as%2014%20mil%C3%B5es,5%25%20das%20pessoas%20sem%20defici%C3%Aancia> . Acesso em: 15 mar. 2026.

JIANG, X. et al. Low-latency networking: **Where latency lurks and how to tame it**. Proceedings of the IEEE, IEEE, v. 107, n. 2, p. 280–306, 2018.

MEDOLA, F. O., GAMA, S., Elui, V., PASCHOARELLI, L., FOTULAN, C. A. **User's perceptions on mobility, comfort and usability of manual wheelchairs.** In Proceedings of the 5th International Conference on Applied Human Factors and Ergonomics (AHFE). Switzerland: Springer, 2014. Disponível em:
<<https://www.researchgate.net/publication/265162312>>. Acesso em: 13 fev. 2026.

MILLER, L. V. **Conditions in occupational therapy: effect on occupational performance.** Baltimore: Lippincott Williams & Wilkins, 2012.

MONTEIRO JÚNIOR, S. **Aplicação do controle de impedância para assistência ao operador de um sistema de movimentação de cargas**. 2016. Disponível em: <<https://repositorio.fei.edu.br/handle/FEI/158>>. Acesso em: 20 jan. 2026.

MORAIS, J. A. **ESP32 – Analisando e corrigindo o ADC interno**. Embarcados, 2026.
Disponível em: <<https://embarcados.com.br/esp32–adc–interno/>>. Acesso em: 10 mar. 2026.

OMS – Organização Mundial de Saúde. **Diretrizes sobre o fornecimento de cadeiras de rodas manuais em locais com poucos recursos**. 2008. Disponível em: http://apps.who.int/iris/bitstream/10665/43960/38/9789241547482_por.pdf. Acesso em: 29/04/2022.

Prabhu. A. **ESP32 Pinout Reference**. Last Minute Engineers, 2026. Disponível em: <<https://lastminuteengineers.com/esp32-pinout-reference/>>. Acesso em: 10 mar. 2026.

QU, Y; et al. **Privacy of Things: Emerging Challenges and Opportunities in Wireless Internet of Things**. IEEE Wireless Communications, v. 25, n. 6, p. 91–97, dez. 2018. doi: 10.1109/MWC.2017.1800112.

SIMPSON, R. C., LOPRESTI, E. F., COOPER, R. A. **How many people would benefit from a smart wheelchair?**, 2008. Journal of Rehabilitation Research and Development,

45(1), 53–72. Disponível em: <<http://dx.doi.org/10.1682/JRRD.2007.01.0015>>. Acesso em: 10 mar. 2026.

SIKIMIĆ, M; et al. **An Overview of Wireless Technologies for IoT Network**. In: INTERNATIONAL SYMPOSIUM INFOTEH–JAHORINA, 19, 18–20 mar. 2020.

SILVA, O. L. **Dedução das Equações de Strain Gauge**. 2022

SILVA JUNIOR, V. P. **Conheça a tecnologia LoRa® e o protocolo LoRaWAN™**. Embarcados, 2016. Disponível em: <https://www.embarcados.com.br/conhecatecnologia-lora-e-o-protocolo-lorawan/#Kits-para-desenvolvimento>. Acesso em: 26 jan. 2026.

SONG, P.; YU, Y.; ZHANG, X. **Impedance control of robots: An overview**. In: 2017 2nd International Conference on Cybernetics, Robotics and Control (CRC). [S.l.: s.n.], 2017. p. 51–55.

SPARKFUN. **nRF24L01_Product_Specification_v2_0**, 2026. Disponível em: <https://cdn.sparkfun.com/datasheets/Wireless/Nordic/nRF24L01_Product_Specification_v2_0.pdf>. Acesso em: 18 mar. 2026.

TEXAS INSTRUMENTS. **CD4052**, 2026. Disponível em: <<https://www.ti.com/product/CD4052B>>. Acesso em: 18 mar. 2026.

TEXAS INSTRUMENTS. **LM358**, 2026. Disponível em: <<https://www.ti.com/lit/ds/symlink/lm358.pdf>>. Acesso em: 18 mar. 2026.

ANEXOS

Código 1 - Leitura de 4 *strain gauges* e conversão ADC utilizando DMA

```
/*
 * Funcao: Leitura de 4 strain gauges e conversão ADC utilizando DMA
 * Autor : Paulo Cesar Menegon de Castro
 * Data de criacao : 20.02.2026
 * Data de modificacao: 11.03.2026
 * Referencias: https://espressif-docs.readthedocs-hosted.com/projects/esp-idf/en/stable/api-reference/peripherals/i2s.html
 *              https://embarcados.com.br/esp32-adc-interno/
 */

// Constants
#define LED 2 // Board LED
#define ADC_UNIT ADC_UNIT_1 // Define ADC unit (ADC1)
#define ADC_CHANNEL ADC1_CHANNEL_7 // Define ADC channel (ADC1_CHANNEL_7 =
GPIO_35)
#define SAMPLE_RATE 128000 // 128KHz
#define MUX_A 15 // Address pins for multiplex
#define MUX_B 14 // MSB

// Libraries
#include <driver/i2s.h>
#include <driver/adc.h>

// Timer0 instantiation
hw_timer_t* timer0 = NULL;

// Variables
bool t0_int = false; // Timer0 interrupt flag
size_t bytes_read; // Number of bytes read from ADC
int samples_read; // Number of samples read
int16_t buffer[128]; // Store values read from ADC
uint16_t sg[4]; // Data from strain gauges. Value read ADC. Demands 2
bytes (0 a 4095)

unsigned long start_chrono; // Start chronometer
unsigned long stop_chrono; // Stop chronometer

// Timer0 interrupt
void IRAM_ATTR timer0_int() { // RAM is faster than FLASH
    t0_int = true;
}
```

```

// Timer0 initialization
void startTimer0() {
    // Frequency of 1 MHz
    timer0 = timerBegin(1000000); // Frequency in Hz (max 80MHz)

    // Timer0 interruption function
    timerAttachInterrupt(timer0, &timer0_int);

    // Function call interval
    timerAlarm(timer0, 1000, true, 0); // Function called every 1000 useconds
    (1KHZ)
}

void i2s_init() {
    i2s_config_t i2s_config = {
        .mode = (i2s_mode_t)(I2S_MODE_MASTER | I2S_MODE_RX |
I2S_MODE_ADC_BUILT_IN), // I2S receive mode with ADC
        .sample_rate =
SAMPLE_RATE, // Sample rate
        .bits_per_sample =
I2S_BITS_PER_SAMPLE_16BIT, // 16 bit I2S
        .channel_format =
I2S_CHANNEL_FMT_ONLY_LEFT, // Only the left
channel
        .communication_format =
I2S_COMM_FORMAT_STAND_I2S, // I2S format
        .intr_alloc_flags =
ESP_INTR_FLAG_LEVEL1, // Default interrupt
priority
        .dma_buf_count =
8, // Number of DMA
buffers
        .dma_buf_len =
64, // Number of
samples
        .use_apll =
false, // No Audio
PLL
        .tx_desc_auto_clear = false,
        .fixed_mclk = 0
    };
    i2s_driver_install(I2S_NUM_0, &i2s_config, 0, NULL);
    i2s_set_adc_mode(ADC_UNIT, ADC_CHANNEL);
    i2s_adc_enable(I2S_NUM_0);
}

```

```

void setup() {
    // Serial port init
    Serial.begin(921600);

    // Onboard LED
    pinMode(LED, OUTPUT);

    // Multiplex address pins
    pinMode(MUX_A, OUTPUT);
    pinMode(MUX_B, OUTPUT);

    // Timer0 start
    startTimer0();

    // i2s start
    i2s_init();

    // ADC channel attenuation
    adc1_config_channel_atten(ADC_CHANNEL, ADC_ATTEN_DB_11); // Allows input up
    to 2.5V-3.3V, but the ADC is most linear between 100mV and 2450mV
}

void loop() {
    // Serial communication causes a lot of delay! Use if only necessary.
    if (t0_int) {
        // Interruption flag
        t0_int = false;

        // Total time for 4 channel reads
        uint16_t total_time_adc = 0;

        // Indicate the start of process
        digitalWrite(LED, HIGH);

        // Number of straining gauge read
        uint8_t sg_number = 0;

        // Read from wheatstone bridges (1 to 4)
        for (int i = 0; i < 2; i++) {
            // Multiplex Control B (MSB)
            digitalWrite(MUX_B, i);

            for (int j = 0; j < 2; j++) {

                // Start ADC reading and processing data from
                start_chrono = micros(); // start of process
            }
        }
    }
}

```

```

        // Multiplex Control A (LSB)
        digitalWrite(MUX_A, j);

        // Read data from DMA
        i2s_read(I2S_NUM_0, &buffer, sizeof(buffer), &bytes_read,
portMAX_DELAY);
        samples_read = bytes_read / 2;

        // Process data and average
        float sum = 0; // Sum of read values from ADC
        for (int i = 0; i < samples_read; i++) {
            // Only upper 12 bits contain the ADC value
            //adc_val = buffer[i] & 0x0FFF;
            sum += buffer[i] & 0x0FFF;
        }
        // Average of read values
        sg[sg_number] = (int)sum / samples_read;
        sg_number++;

        // Indicate the end of process
        stop_chrono = micros();

        // Results
        Serial.println("-----");
        Serial.print("Time for reading and processing data from strain gauge :");
        Serial.print(sg_number);
        Serial.print(" (us): ");
        Serial.println(stop_chrono - start_chrono);
        total_time_adc = total_time_adc + (stop_chrono - start_chrono);
    }
}

// Indicate the end of conversion
digitalWrite(LED, LOW);

// Results
Serial.println("-----");
Serial.print("Time for reading and processing data from all strain gauges : ");
Serial.print(total_time_adc);
Serial.println(" (us): ");
Serial.println("-----");
Serial.println("Valor medio das amostras: ");

```



```
Serial.print("Strain gauge 1: ");  
Serial.println(sg[0]);  
Serial.print("Strain gauge 2: ");  
Serial.println(sg[1]);  
Serial.print("Strain gauge 3: ");  
Serial.println(sg[2]);  
Serial.print("Strain gauge 4: ");  
Serial.println(sg[3]);  
}  
}
```

Código 2 - Transmissão de pacotes contendo 4 valores aleatórios de 0 a 4095 (simulando leituras dos *strain gauges*)

```
/*
 * Funcao: Transmissao de pacotes contendo 4 valores aleatorios de 0 a 4095
 (simulando leituras dos strain gauges)
 * Autor: Paulo Cesar Menegon de Castro
 * Data de criacao: 20.02.2026
 * Data de modificacao: 11.03.2026
 *
 * Referencias: https://randomnerdtutorials.com/esp32-spi-communication-arduino/
               https://randomnerdtutorials.com/nrf24l01-2-4ghz-rf-transceiver-module-with-arduino/
 *
 * -----
 * | Pino do Modulo | Pino ESP32 |
 * -----
 * | GND            | GND        |
 * -----
 * | 3.3V           | 3.3V        |
 * -----
 * | CE             | 4           |
 * -----
 * | CSN            | 5           |
 * -----
 * | SCK            | 18          |
 * -----
 * | MOSI           | 23          |
 * -----
 * | MISO           | 19          |
 * -----
 * | IRQ            | 22          |
 * -----
 */

// Constants
#define LED 33      // GPIO_33
#define CE_PIN 4    // CE nRF24L01
#define CSN_PIN 5   // CSN nRF24L01

// Libraries
#include <SPI.h>
#include <RF24.h>
#include <printf.h>
```

```

// Timer0 instantiation
hw_timer_t* timer0 = NULL;

// Instantiate an object for the nRF24L01 transceiver
RF24 radio(CE_PIN, CSN_PIN);

// Create a struct
struct payloadStruct {
    unsigned long time_stamp; // Milliseconds since board is on
    unsigned long number = 0; // Number of payload
    uint16_t sg[4];           // Data from strain gauges. Value read ADC.
    Demands 2 bytes (0 a 4095)
};
payloadStruct payload;

// Variables
bool t0_int = false;           // Timer0 interrupt flag
unsigned tx_irq = 0;           // Number of interruptions requests to
transmission
unsigned tx_failure = 0;       // Number of transmissions failure
unsigned long start_chrono;    // Start of a process
unsigned long stop_chrono;     // End of process
unsigned long last_time_stamp = 0;

// Let these addresses be used for the pair
uint8_t address[][4] = { "1No", "2No" };
// It is very helpful to think of an address as a path instead of as
// an identifying device destination
// to use different addresses on a pair of radios, we need a variable to
// uniquely identify which address this radio will use to transmit

// Timer0 interrupt
void IRAM_ATTR timer0_int() { // RAM is faster than FLASH
    t0_int = true;
}

// Timer0 initialization
void startTimer0() {
    // Frequency of 1 MHz
    timer0 = timerBegin(1000000); // Frequency in Hz (max 80MHz)

    // Timer0 interruption function
    timerAttachInterrupt(timer0, &timer0_int);

    // Function call interval
    timerAlarm(timer0, 1000, true, 0); // Function called every 1000 useconds
(1KHZ)

```

```

}

void setup() {
  // Serial port init
  Serial.begin(921600);

  // Onboard LED
  pinMode(LED, OUTPUT);

  // Timer0 start
  startTimer0();

  // Start nRF24L01
  if (!radio.begin()) {
    Serial.println(F("radio hardware is not responding!!"));
    while (1) {} // hold in infinite loop
  }

  // Set the PA Level low to try preventing power supply related problems
  radio.setPALevel(RF24_PA_LOW); // RF24_PA_MAX is default.
  radio.setDataRate(RF24_2MBPS); // MAX speed
  radio.setAutoAck(false); // Disable ACK
  radio.disableCRC(); // Optional (fastest, less safe)
  radio.disableDynamicPayloads(); // Disable dynamic payloads
  radio.disableAckPayload(); // Disable ACK payloads
  radio.disableCRC(); // Optional (fastest, less safe)
  radio.setRetries(0, 0); // No retransmissions
  radio.setRetries(0, 0);
  radio.setChannel(100); // Less WiFi interference
  radio.setAddressWidth(3); // Short address

  // Save on transmission time by setting the radio to only transmit the
  // number of bytes we need to transmit a uint16_t
  radio.setPayloadSize(sizeof(payload)); // int datatype occupies 2 bytes

  // set the TX address of the RX node for use on the TX pipe (pipe 0)
  radio.stopListening(address[0]); // put radio in TX mode

  // set the RX address of the TX node into a RX pipe
  //radio.openReadingPipe(1, address[1]); // using pipe 1

  // TX role
  radio.stopListening(); // put radio in TX mode

  // For debugging info
  printf_begin(); // needed only once for printing details

```

```

    radio.printPrettyDetails(); // (larger) function that prints human readable
data
    //Serial.end();
}

void loop() {
    // Serial communication causes a lot of delay! Use if only necessary.

    if (t0_int) {
        // Interruption flag
        t0_int = false;

        // Interruption requests
        tx_irq++;

        // Fill the payload
        payload.time_stamp = micros(); // Microseconds since reset or board on
        for (int i=0;i<4;i++){
            payload.sg[i]=random(4095); // Random numbers between 0 and 4095
        }

        // Start transmission
        start_chrono = micros(); // start the timer

        // Transmit & save the report
        bool report = radio.writeFast(&payload, sizeof(payload));

        // End of transmission
        stop_chrono = micros();

        // If transmission successfull
        if (report) {
            // Indicate the start of transmission
            digitalWrite(LED, HIGH);
            //Serial.println("-----");
            //Serial.println("Transmission successful! "); // Payload was delivered
            //Serial.print("Time to transmit: ");
            //Serial.println(stop_chrono - start_chrono); // Print the timer result
            //Serial.print("Time stamp (us): ");
            //Serial.println(payload.time_stamp); // Print the time stamp
            //Serial.print("Diference between time stamps (us): ");
            //Serial.println(payload.time_stamp - last_time_stamp); // Print the
time stamp
            //last_time_stamp = payload.time_stamp;
            //Serial.print("Data: ");
            //Serial.print("SG_1: ");
            //Serial.print(payload.sg[0]);

```

```

        //Serial.print(" SG_2: ");
        //Serial.print(payload.sg[1]);
        //Serial.print(" SG_3: ");
        //Serial.print(payload.sg[2]);
        //Serial.print(" SG_4: ");
        //Serial.println(payload.sg[3]);
        //Serial.print("Payload number: ");
        //Serial.print(payload.number);
        payload.number++;
    } else {
        tx_failure++;

        // Retransmission of data lost
        radio.writeFast(&payload, sizeof(payload));
        //Serial.println("-----");
        //Serial.println("Transmission failed or timed out"); // Payload was
not delivered
    }
    // Indicate end of transmission
    digitalWrite(LED, LOW);
}
/*
if (tx_irq == 1000) {
    timerStop(timer0);
    Serial.println("-----");
    Serial.print("Number of interruptions: ");
    Serial.println(tx_irq);
    Serial.print("Number of transmissions: ");
    Serial.println(payload.number);
    Serial.print("Number of failures of transmission: ");
    Serial.println(tx_failure);
    tx_irq = 1001;
}
*/
}

```

Código 3 - Recepção dos pacotes transmitidos

```
/*
 * Funcao: Recepcao dos pacotes transmitidos
 * Autor: Paulo Cesar Menegon de Castro
 * Data de criacao: 20.02.2026
 * Data de modificacao: 11.03.2026
 *
 * Referencias: https://randomnerdtutorials.com/nrf24l01-2-4ghz-rf-transceiver-module-with-arduino/
 *
 * -----
 * | Pino do Modulo | Pino ESP32 |
 * -----
 * | GND | GND |
 * -----
 * | 3.3V | 3.3V |
 * -----
 * | CE | 4 |
 * -----
 * | CSN | 5 |
 * -----
 * | SCK | 18 |
 * -----
 * | MOSI | 23 |
 * -----
 * | MISO | 19 |
 * -----
 * | IRQ | 22 |
 * -----
 */

// Constants
#define LED 2 // Onboard LED
#define CE_PIN 4
#define CSN_PIN 5
#define IRQ 22

// Libraries
#include <SPI.h>
#include <RF24.h>
#include <printf.h>

// Instantiate an object for the nRF24L01 transceiver
RF24 radio(CE_PIN, CSN_PIN);

// Create a struct
struct payloadStruct {
```

```

    unsigned long time_stamp; // Milliseconds since board is on
    unsigned long number = 0; // Number of payload
    uint16_t sg[4];           // Data from strain gauges. Value read ADC.
    Demands 2 bytes (0 a 4095)
};
payloadStruct payload;

// Variables
unsigned long start_chrono; // Start of a process
unsigned long stop_chrono;  // End of process
unsigned long last_time_stamp = 0;

// Let these addresses be used for the pair
uint8_t address[][4] = { "1No", "2No" };
// It is very helpful to think of an address as a path instead of as
// an identifying device destination

// For this example, we'll be using a payload containing
// a single float number that will be incremented
// on every successful transmission
volatile int count = 0; // tracks the number of interrupts from IRQ
bool intNRF24L01 = false; // Flag de interrupcao Timer0

void IRAM_ATTR isrReceive() {
    intNRF24L01 = true;
}

void setup() {
    // Serial begin
    Serial.begin(921600);

    // Onboard LED
    pinMode(LED, OUTPUT);

    // initialize the transceiver on the SPI bus
    if (!radio.begin()) {
        Serial.println(F("radio hardware is not responding!!"));
        while (1) {} // hold in infinite loop
    }

    // Set the PA Level low to try preventing power supply related problems
    // because these examples are likely run with nodes in close proximity to
    // each other.
    radio.setPALevel(RF24_PA_LOW); // RF24_PA_MAX is default.
    radio.setDataRate(RF24_2MBPS); // MAX speed
    radio.setAutoAck(false);        // Disable ACK
    radio.disableDynamicPayloads(); // Disable dynamic payloads

```



```

radio.disableAckPayload();      // Disable ACK payloads
radio.disableCRC();             // Optional (fastest, less safe)
radio.setRetries(0, 0);         // No retransmissions
radio.setChannel(100);          // Less WiFi interference
radio.setAddressWidth(3);       // Short address

// save on transmission time by setting the radio to only transmit the
// number of bytes we need to transmit a int
radio.setPayloadSize(sizeof(payload)); // int datatype occupies 2 bytes

// set the TX address of the RX node for use on the TX pipe (pipe 0)
//radio.stopListening(address[1]); // put radio in TX mode

// set the RX address of the TX node into a RX pipe
radio.openReadingPipe(1, address[0]); // using pipe 1

// RX mode
radio.startListening(); // put radio in RX mode

//mask all IRQ triggers except for receive (1 is mask, 0 is no mask)
radio.maskIRQ(1, 1, 0);

//Create interrupt: 0 for pin 2 or 1 for pin 3, the name of the interrupt
function or ISR, and condition to trigger interrupt
attachInterrupt(IRQ, isrReceive, FALLING);

// For debugging info
printf_begin(); // Needed only once for printing details
radio.printPrettyDetails(); // (larger) function that prints human readable
data
//Serial.end();
}

void loop() {
  // Use of serial communication delay reception rates
  if (intNRF24L01) {
    // Interruption flag
    intNRF24L01 = false;

    uint8_t pipe;
    if (radio.available(&pipe)) { // Is there a payload? get the pipe number
that received it

      // If a payload received
      digitalWrite(LED, HIGH);

      // Start of reception

```

```

start_chrono = micros();

uint8_t bytes = radio.getPayloadSize(); // get the size of the payload
radio.read(&payload, sizeof(payload)); // fetch payload from FIFO

// End of reception
stop_chrono = micros();

//Serial.println("-----");
//Serial.println("Reception successful! "); // Payload was received
//Serial.print("Time to receive: ");
//Serial.println(stop_chrono - start_chrono); // Print the timer result
//Serial.print("Time stamp (us): ");
//Serial.println(payload.time_stamp); // Print the time stamp
//Serial.print("Diference between time stamps (us): ");
//Serial.println(payload.time_stamp - last_time_stamp); // Print the
time stamp
//last_time_stamp = payload.time_stamp;
//Serial.print("Data: ");
//Serial.print("SG_1: ");
//Serial.print(payload.sg[0]);
//Serial.print(" SG_2: ");
//Serial.print(payload.sg[1]);
//Serial.print(" SG_3: ");
//Serial.print(payload.sg[2]);
//Serial.print(" SG_4: ");
//Serial.println(payload.sg[3]);
//Serial.print("Payload number: ");
//Serial.println(payload.number);
//Serial.println();
digitalWrite(LED, LOW);
} else {
//Serial.println("-----");
//Serial.println("Transmission failed or timed out"); // Payload was
not delivered
}
}
}

```